

# PROGRAMMIERUNG 2

## Kapitel 5: **Lambda-Ausdrücke**



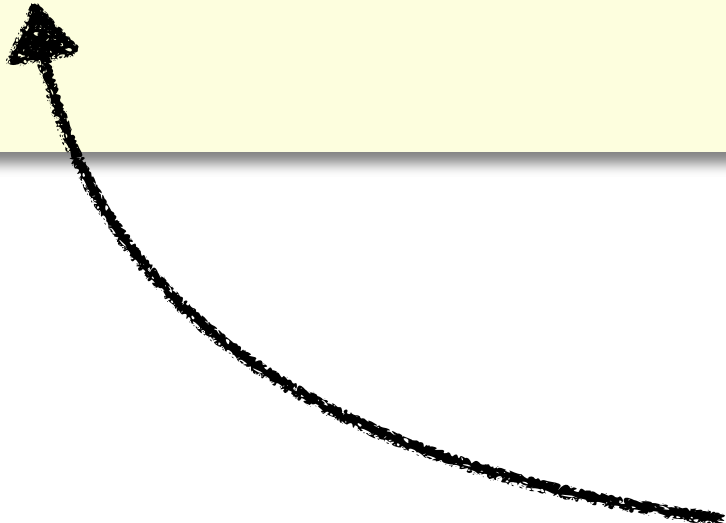
Prof. Dr. Markus Esch  
htw saar

# Rückblick

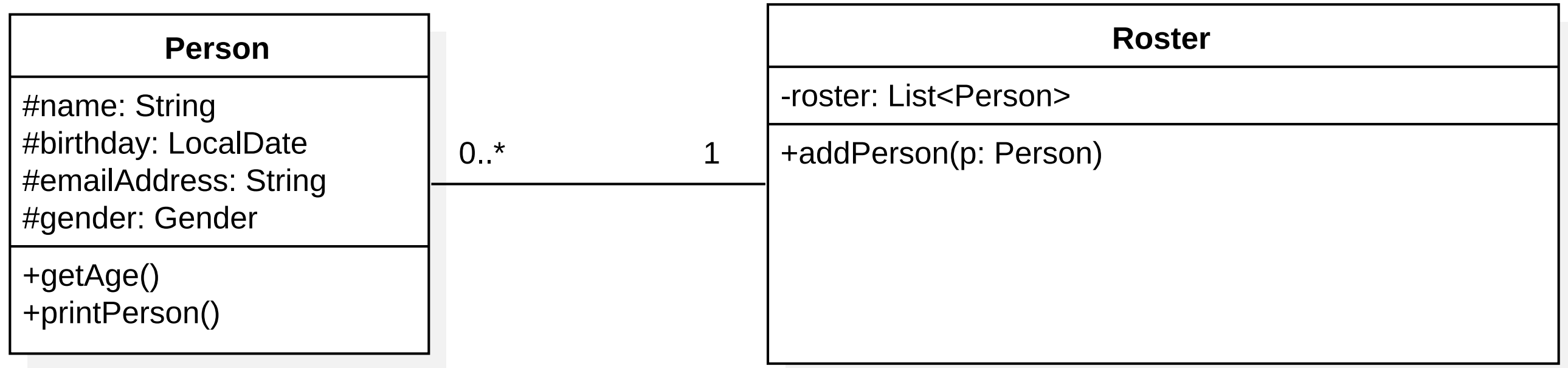
## Anonymous Classes

```
juggleWithNumbers(new Handler() {  
    public void handle(double value) {  
        System.out.println(value);  
    }  
});
```

Übergabe von  
Funktionalität als  
Argument an eine  
Methode



# Motivierendes **Beispiel**



# Motivierendes **Beispiel**

```
public class Person {  
    public enum Gender {MALE, FEMALE}  
    String name;  
    LocalDate birthday;  
    String emailAddress;  
    Gender gender;  
  
    public int getAge() {  
        return birthday  
            .until(IsoChronology.INSTANCE.dateNow())  
            .getYears();  
    }  
    public void printPerson() {  
        System.out.println(name + ", "  
            + this.getAge());  
    }  
    ...  
}
```

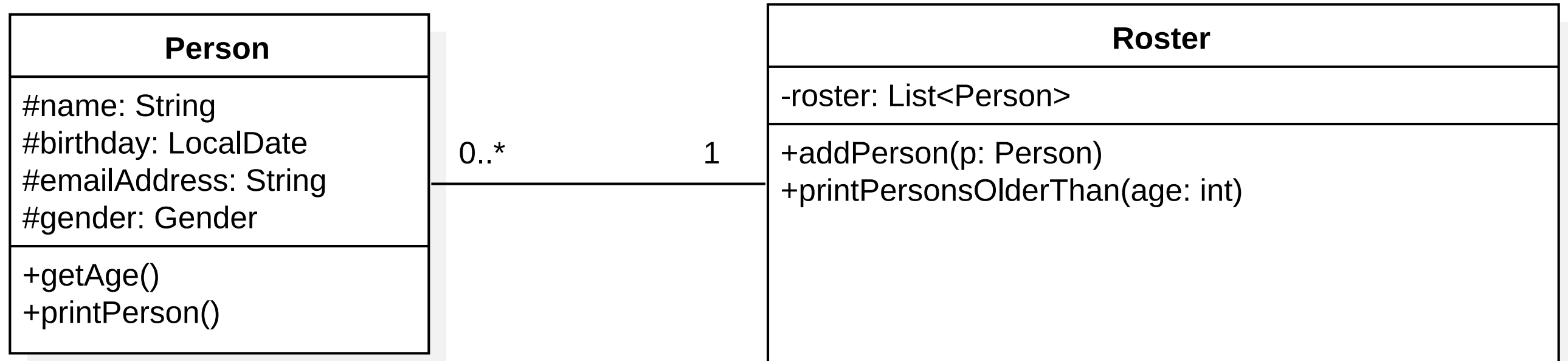


# Motivierendes **Beispiel**

```
public class Roster {  
    List<Person> roster;  
  
    public Roster(){  
        roster = new ArrayList<Person>();  
    }  
  
    public void addPerson(Person p)  
    {  
        this.roster.add(p);  
    }  
}
```

*Aufgabe:  
Ausgabe aller  
Personen älter  
als x*

# Motivierendes **Beispiel**

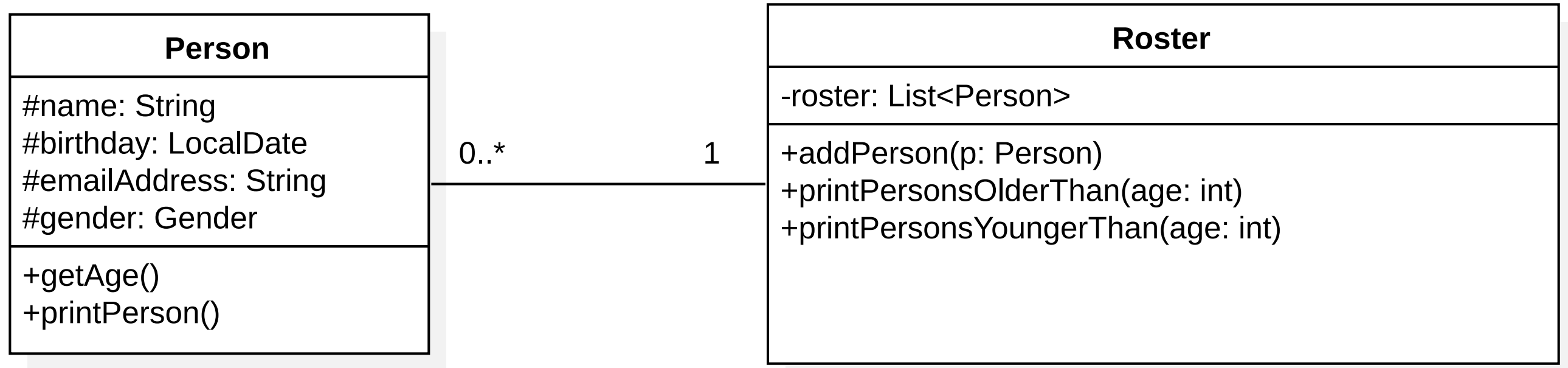


# Motivierendes **Beispiel**

```
public void printPersonsOlderThan(int age) {  
    for (Person p : this.roster) {  
        if (p.getAge() >= age) {  
            p.printPerson();  
        }  
    }  
}
```

Aufgabe: Ausgabe aller  
Personen jünger als x

# Motivierendes **Beispiel**



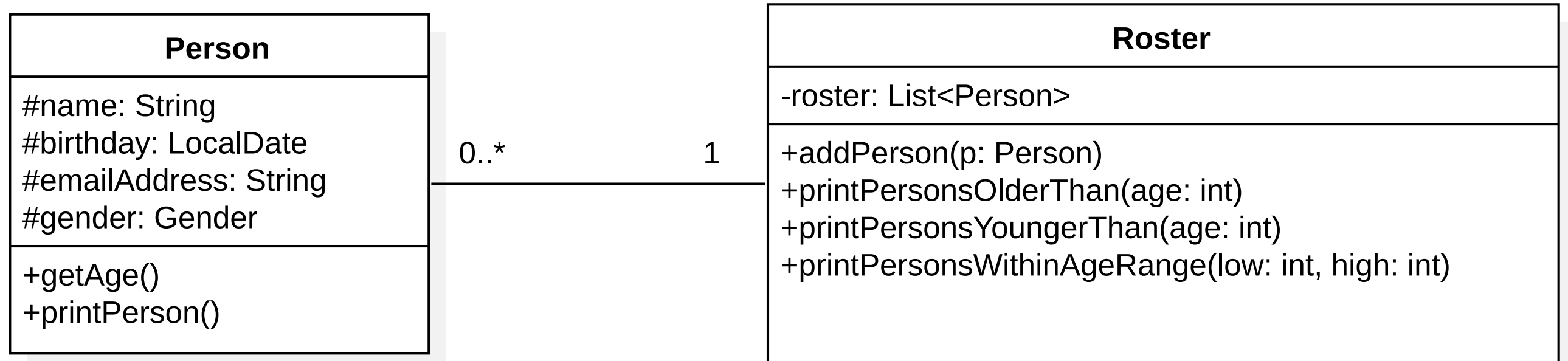


# Motivierendes **Beispiel**

```
public void printPersonsYoungerThan(int age) {  
    for (Person p : this.roster) {  
        if (p.getAge() < age) {  
            p.printPerson();  
        }  
    }  
}
```

# Motivierendes **Beispiel**

## ► Allgemeinere Lösung



# Motivierendes **Beispiel**

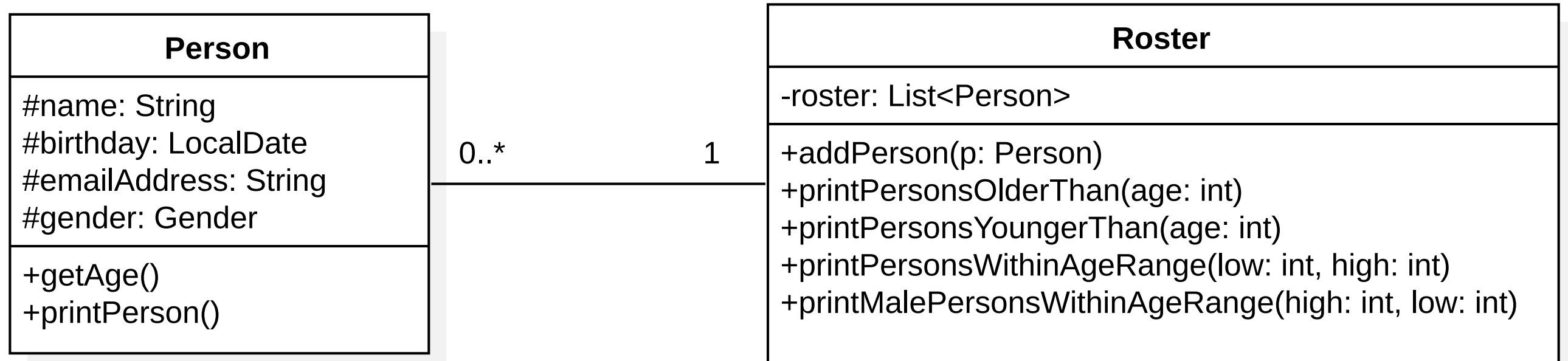
## ► Allgemeinere Lösung

```
public void printPersonsWithinAgeRange(  
    int low, int high) {  
  
    for (Person p : roster) {  
        if (low <= p.getAge() && p.getAge() < high) {  
            p.printPerson();  
        }  
    }  
}
```

# Motivierendes **Beispiel**

Aufgabe:  
Ausgabe aller  
männlichen  
Personen älter  
als  $x$

# Motivierendes **Beispiel**



# Motivierendes **Beispiel**

```
public void printMalePersonsWithinAgeRange(  
    int low, int high) {  
  
    for (Person p : roster) {  
        if (low <= p.getAge() && p.getAge() < high &&  
            p.gender == Person.Gender.MALE) {  
  
            p.printPerson();  
  
        }  
    }  
}
```



# Motivierendes **Beispiel**

Aufgabe:  
Ausgabe aller  
weiblichen  
Personen älter  
als x deren  
Name mit "K"  
beginnt



Welche **andere**  
Herangehensweise wäre möglich?



# Motivierendes **Beispiel**

- Auslagerung der Test-Kriterien

```
interface CheckPerson {  
    boolean test(Person p);  
}
```

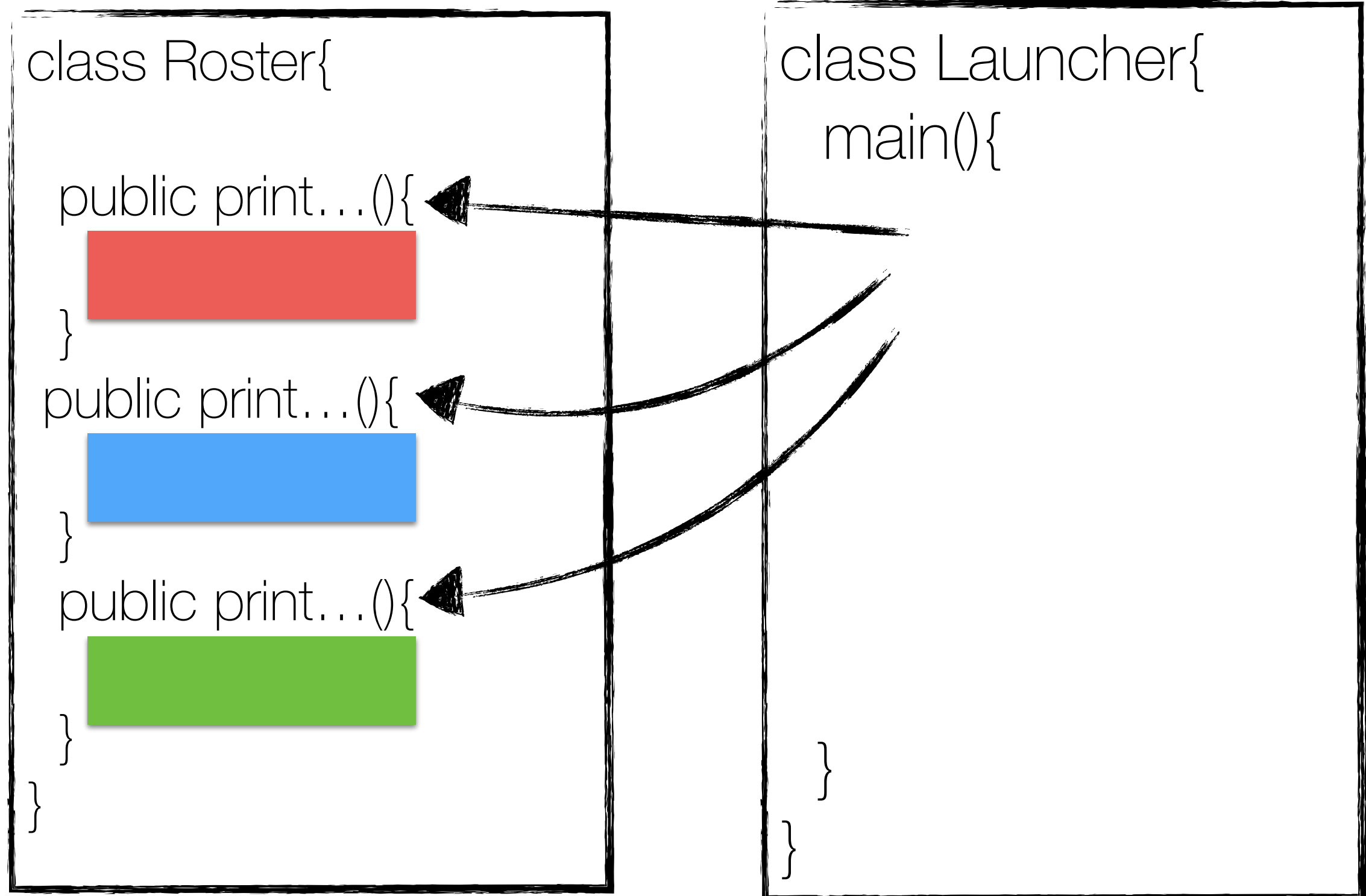
```
public void printPersons(CheckPerson tester) {  
    for (Person p : roster) {  
        if (tester.test(p)) {  
            p.printPerson();  
        }  
    }  
}
```

# Motivierendes **Beispiel**

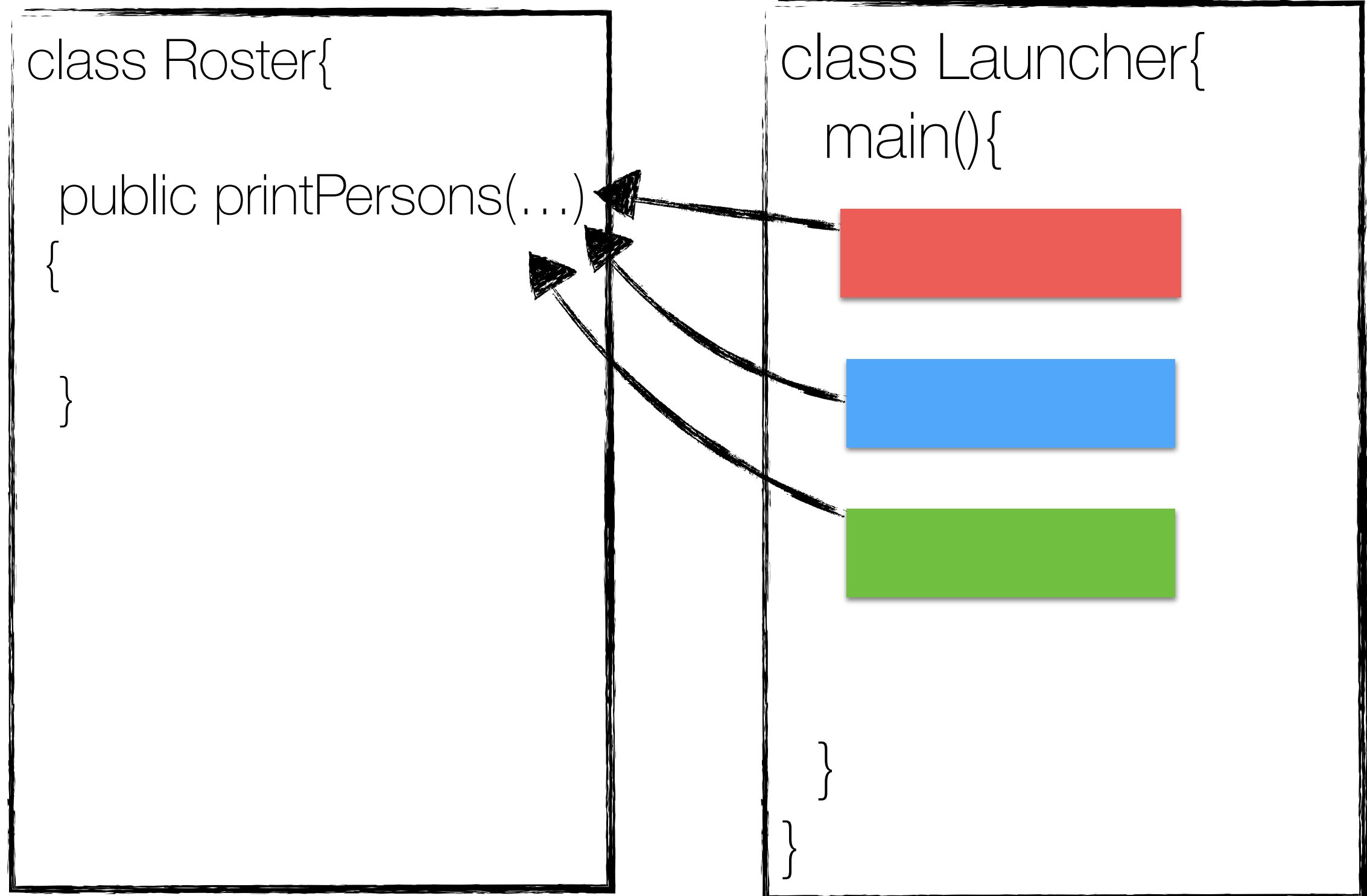
- Aufruf mit lokaler Klasse

```
public static void main(String[] args)
{
    Roster roster = new Roster();
    class CheckAgeAndGender implements CheckPerson {
        public boolean test(Person p) {
            return p.gender == Person.Gender.MALE &&
                p.getAge() >= 18 &&
                p.getAge() <= 25;
        }
    }
    CheckAgeAndGender checker = new CheckAgeAndGender();
    roster.printPersons(checker);
}
```

# Motivierendes **Beispiel**

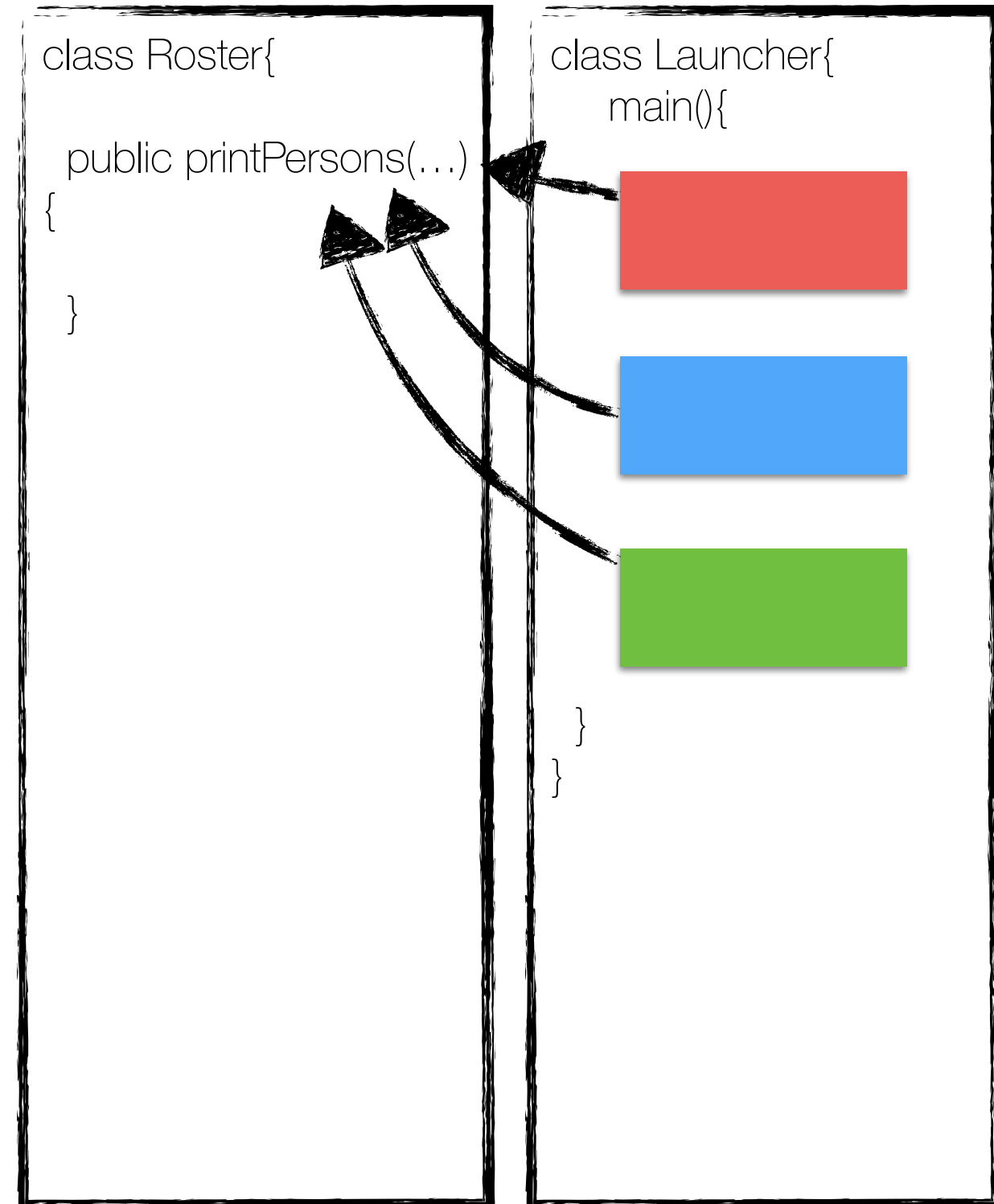
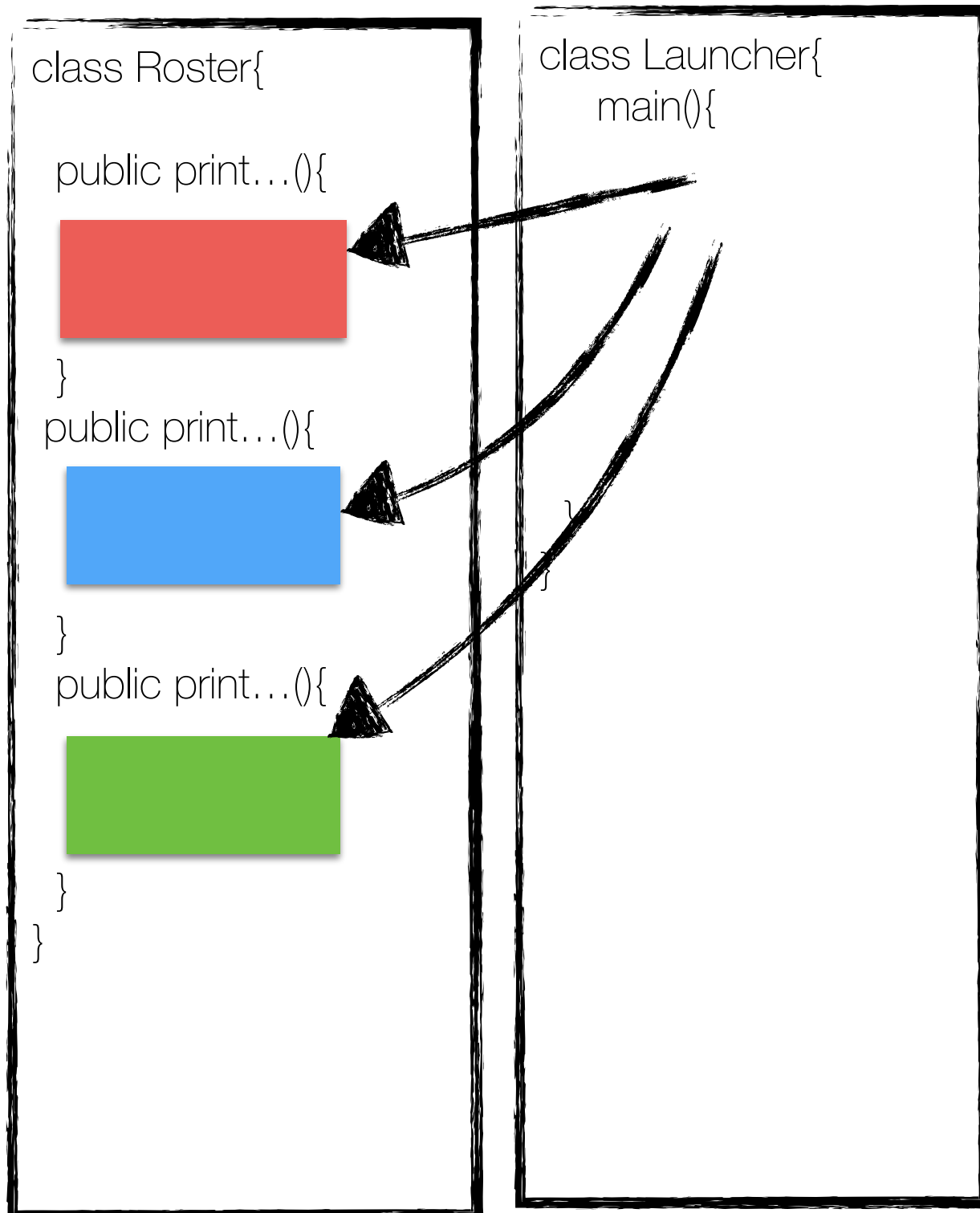


# Motivierendes **Beispiel**





# Motivierendes **Beispiel**

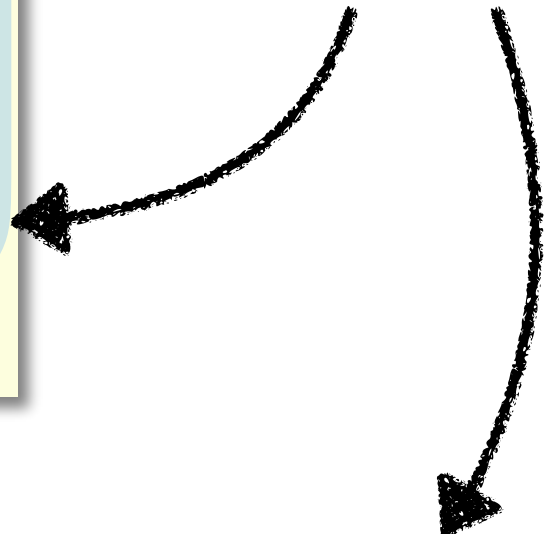


# Motivierendes **Beispiel**

- Noch einfacher: Aufruf mit anonymer Klasse

```
Roster roster = new Roster();
roster.printPersons(
    new CheckPerson() {
        public boolean test(Person p) {
            return p.getGender() == Person.Gender.MALE
                && p.getAge() >= 18
                && p.getAge() <= 25;
        }
    }
);
```

Übergabe von  
**Funktionalität** als  
Argument an eine  
Methode



Konzept **funktionaler**  
Programmierung!!!

# Lambda-Ausdrücke

seit Java **8**



**Funktionale** Konzepte in Java

# Motivierendes **Beispiel**

```
Roster roster = new Roster();
roster.printPersons(
    new CheckPerson() {
        public boolean test(Person p) {
            return p.getGender() == Person.Gender.MALE
                && p.getAge() >= 18
                && p.getAge() <= 25;
        }
    }
);
```

Anonyme  
Klasse

```
roster.printPersons(
    (Person p) -> p.getGender() == Person.Gender.MALE
        && p.getAge() >= 18
        && p.getAge() <= 25
);
```

Lambda  
Ausdruck

# Motivierendes **Beispiel**

```
Roster roster = new Roster();
roster.printPersons(
    new CheckPerson() {
        public boolean test(Person p) {
            return p.getGender() == Person.Gender.MALE
                && p.getAge() >= 18
                && p.getAge() <= 25;
        }
    }
);
```

Anonyme  
Klasse

```
roster.printPersons(
    (Person p) -> p.getGender() == Person.Gender.MALE
        && p.getAge() >= 18
        && p.getAge() <= 25
);
```

Lambda  
Ausdruck



# Motivierendes **Beispiel**

```
Roster roster = new Roster();
roster.printPersons(
    new CheckPerson() {
        public boolean test(Person p) {
            return p.getGender() == Person.Gender.MALE
                && p.getAge() >= 18
                && p.getAge() <= 25;
        }
    }
);
```

Anonyme  
Klasse

```
roster.printPersons(
    (Person p) -> p.getGender() == Person.Gender.MALE
        && p.getAge() >= 18
        && p.getAge() <= 25
);
```

Lambda  
Ausdruck

# Motivierendes **Beispiel**

```
Roster roster = new Roster();
roster.printPersons(
    new CheckPerson() {
        public boolean test(Person p) {
            return p.getGender() == Person.Gender.MALE
                && p.getAge() >= 18
                && p.getAge() <= 25;
        }
    }
);
```

Anonyme  
Klasse

```
roster.printPersons(
    (Person p) -> p.getGender() == Person.Gender.MALE
        && p.getAge() >= 18
        && p.getAge() <= 25
);
```

Lambda  
Ausdruck



# Einführung **Lambda-Ausdrücke**

- ▶ Lambdas stammen aus funktionalen Programmiersprachen
  - ▶ anonyme Methoden
    - ▶ ad-hoc Implementierung von **Funktionalität**
- ▶ **Code as Data**
  - ▶ Funktionalität als Parameter/Rückgabewert weitergeben
- ▶ Überlegen gegenüber (anonymen) inneren Klassen
  - ▶ prägnante Syntax, weniger Code, lesbarer...
    - ▶ ...“funktionaler“!

# Lambda-Ausdrücke **Syntax**

```
lambda = ParameterList -> Body
```

```
ParameterList = Identifier
```

```
| "(" Identifier [ "," Identifier ]* ")"
```

```
| "(" Type Identifier [ "," Type Identifier ]* ")"
```

```
Body = Expression
```

```
| "{" [ Statement ";" ]+ "}"
```

- ▶ Rückgabetyt wird nie angegeben, sondern immer deduziert
- ▶ Argumenttypen können ebenfalls weggelassen werden

# Lambda-Ausdrücke **Syntax**

```
roster.printPersons(  
    (Person p) -> p.getGender() == Person.Gender.MALE  
    && p.getAge() >= 18  
    && p.getAge() <= 25  
);
```

```
roster.printPersons(  
    p -> p.getGender() == Person.Gender.MALE  
    && p.getAge() >= 18  
    && p.getAge() <= 25  
);
```

```
roster.printPersons(  
    p -> {  
        return p.getGender() == Person.Gender.MALE  
        && p.getAge() >= 18  
        && p.getAge() <= 25;  
    }  
);
```

Implizites  
return bei  
einem  
Ausdruck

# Lambda-Ausdrücke **Beispiele**

```
(int x, int y) -> { return x + y; }
```

Argumenttypen  
abgeleitet

```
(x, y) -> { return x + y; }
```

keine Klammern  
notwendig bei  
einem Argument

```
x -> { return x + 1; }
```

Implizites return

```
x -> x + 1;
```

Ausdruck ohne  
Parameter

```
() -> { System.out.println("Hello World"); }
```



# Lambda-Ausdrücke **Beispiele**

```
a -> {  
  if(a<10)  
    return true;  
  else  
    return false;  
}
```

Lambda-  
Ausdruck mit  
Statement-Block



```
a -> (a<10) ? true : false
```

Lambda-  
Ausdruck mit  
einzelter  
Expression





# Herausforderung **funktionale** Konzepte in **OO**-Sprache

Konsistenz

Abwärtskompatibilität

Verständlichkeit

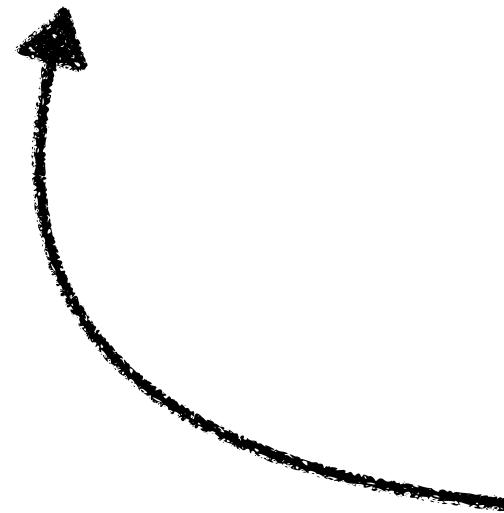
# Funktionale Interfaces

- ▶ Realisierung von Lambdas durch funktionale Interfaces
  - ▶ Single-Abstract-Method-Typen (SAM-Typen)
- ▶ Funktionales Interface  $\Leftrightarrow \exists!$  abstrakte Methode

# Funktionale Interfaces

- ▶ Realisierung von Lambdas durch funktionale Interfaces
  - ▶ Single-Abstract-Method-Typen (SAM-Typen)
- ▶ Funktionales Interface  $\Leftrightarrow \exists!$  **abstrakte Methode**

Sind nicht alle  
Interface-Methoden  
abstrakt???



# Default-Methoden

► ebenfalls neu seit **Java 8**

```
public interface Rectangle {  
    double getWidth();  
  
    double getHeight();  
  
    default boolean isSquare() {  
        return getWidth() == getHeight();  
    }  
}
```

Interface kann  
default  
Implementierung  
angeben



keyword



# Default-Methoden

```
public class Square implements Rectangle{  
    double size;  
  
    public double getWidth() {  
        return size;  
    }  
  
    public double getHeight() {  
        return size;  
    }  
  
    public boolean isSquare(){  
        return true;  
    }  
}
```

Default-  
Methoden  
können  
überschrieben  
werden





# Funktionale Interfaces

- ▶ Realisierung von Lambdas durch funktionale Interfaces
  - ▶ Single-Abstract-Method-Typen (SAM-Typen)
- ▶ Funktionales Interface  $\Leftrightarrow \exists!$  **abstrakte Methode**

# Funktionale Interfaces

- ▶ Realisierung von Lambdas durch funktionale Interfaces
  - ▶ Single-Abstract-Method-Typen (SAM-Typen)
- ▶ Funktionales Interface  $\Leftrightarrow \exists!$  **abstrakte Methode**
- ▶ Lambda-Ausdrücke sind Implementierungen von Funktionalen Interfaces
  - ▶ d.h. ein Lambda-Ausdruck implementiert die abstrakte Methode
  - ▶ Name des Funktionalen Interfaces und der Methode irrelevant
- ▶ Typdeduktion notwendig
  - ▶ Parametertypen, Returntyp

# Funktionale Interfaces

```
public interface Handler {  
    public void handle(double value);  
}
```

```
public void juggleWithNumbers(Handler handler) {  
    double values[] = { 1.0, 5.0, 3.0 };  
    for (double v : values) {  
        handler.handle(v);  
    }  
}
```

# Funktionale Interfaces

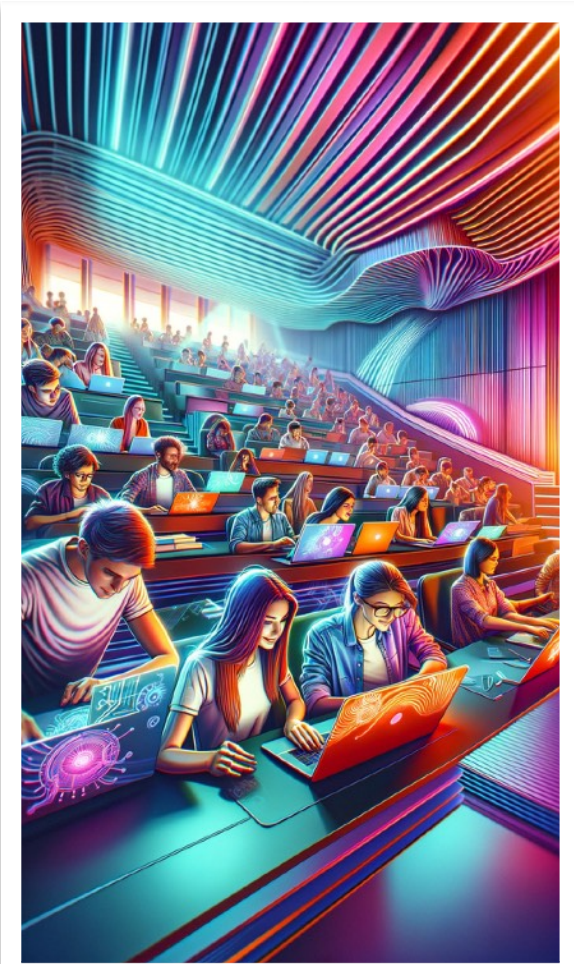
```
public interface Handler {  
    public void handle(double value);  
}
```

```
public void juggleWithNumbers(Handler handler) {  
    double values[] = { 1.0, 5.0, 3.0 };  
    for (double v : values) {  
        handler.handle(v);  
    }  
}
```

Aufgabe: Rufen Sie die Methode mit verschiedenen Handlern auf, die folgendes machen:

- Ausgabe der jeweiligen Werte
- Ausgabe der Quadratzahlen der Werte
- Ausgabe aller Zahlen kleiner 4

Implementieren Sie die Handler jeweils als **anonyme** Klasse und als **Lambda**-Ausdruck



Templates

# Funktionale Interfaces

```
public interface Handler {  
    public void handle(double value);  
}
```

```
Handler anonymousClass = new Handler() {  
    public void handle(double value) {  
        System.out.println(value);  
    }  
};
```

Implementierung als  
anonyme Klasse

```
Handler lambda1 = (double value) -> {  
    System.out.println(value);  
};
```

Implementierung als  
Lambda-Ausdruck

```
Handler lambda2 = value -> {  
    System.out.println(value);  
};
```

impliziter Argumenttyp

```
Handler lambda3 = value ->  
System.out.println(value);
```

nur ein Ausdruck

# Funktionale Interfaces

```
public interface Handler {  
    public void handle(double value);  
}
```

```
public void juggleWithNumbers(Handler handler) {  
    double values[] = { 1.0, 5.0, 3.0 };  
    for (double v : values) {  
        handler.handle(v);  
    }  
}
```

```
juggleWithNumbers(anonymousClass);
```

```
juggleWithNumbers(lambda1);
```

# Lambda-Ausdrücke **Beispiel**



- **Juggler.java**
- **DataStructure.java**
- **Calculator.java**



# **Funktionale** Interfaces

- ▶ Lambda-Ausdrücke werden in Instanzen von Funktionalen Interfaces umgewandelt
  - ▶ Name des Funktionalen Interfaces und der Methode irrelevant

# Funktionale Interfaces

- ▶ Lambda-Ausdrücke sind Implementierungen von Funktionalen Interfaces
  - ▶ Name des Funktionalen Interfaces und der Methode irrelevant



45 Sekunden

# java.util.**function**

- ▶ Sammlung von Funktionalen Interfaces
- ▶ Definition verschiedener Typen (Namenskonventionen)
  - ▶ **Consumer**: Kein Resultat
  - ▶ **Function**: Produziert Resultat
  - ▶ **Operator**: Produziert Resultat vom Argument-Typ
  - ▶ **Supplier**: Produziert Resultat ohne Argument
  - ▶ **Predicate**: Produziert boolean-Resultat

# java.util.function

## Interface Summary

| Interface                   | Description                                                                                                      |
|-----------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>BiConsumer</b> <T,U>     | Represents an operation that accepts two input arguments and returns no result.                                  |
| <b>BiFunction</b> <T,U,R>   | Represents a function that accepts two arguments and produces a result.                                          |
| <b>BinaryOperator</b> <T>   | Represents an operation upon two operands of the same type, producing a result of the same type as the operands. |
| <b>BiPredicate</b> <T,U>    | Represents a predicate (boolean-valued function) of two arguments.                                               |
| <b>BooleanSupplier</b>      | Represents a supplier of boolean-valued results.                                                                 |
| <b>Consumer</b> <T>         | Represents an operation that accepts a single input argument and returns no result.                              |
| <b>DoubleBinaryOperator</b> | Represents an operation upon two double-valued operands and producing a double-valued result.                    |
| <b>DoubleConsumer</b>       | Represents an operation that accepts a single double-valued argument and returns no result.                      |
| <b>DoubleFunction</b> <R>   | Represents a function that accepts a double-valued argument and produces a result.                               |
| <b>DoublePredicate</b>      | Represents a predicate (boolean-valued function) of one double-valued argument.                                  |
| <b>DoubleSupplier</b>       | Represents a supplier of double-valued results.                                                                  |
| <b>DoubleToIntFunction</b>  | Represents a function that accepts a double-valued argument and produces an int-valued result.                   |
| <b>DoubleToLongFunction</b> | Represents a function that accepts a double-valued argument and produces a long-valued result.                   |
| <b>DoubleUnaryOperator</b>  | Represents an operation on a single double-valued operand that produces a double-valued result.                  |
| <b>Function</b> <T,R>       | Represents a function that accepts one argument and produces a result.                                           |
| <b>IntBinaryOperator</b>    | Represents an operation upon two int-valued operands and producing an int-valued result.                         |
| <b>IntConsumer</b>          | Represents an operation that accepts a single int-valued argument and returns no result.                         |
| <b>IntFunction</b> <R>      | Represents a function that accepts an int-valued argument and produces a result.                                 |

# java.util.function

|                                   |                                                                                                            |
|-----------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>IntPredicate</b>               | Represents a predicate (boolean-valued function) of one int-valued argument.                               |
| <b>IntSupplier</b>                | Represents a supplier of int-valued results.                                                               |
| <b>IntToDoubleFunction</b>        | Represents a function that accepts an int-valued argument and produces a double-valued result.             |
| <b>IntToLongFunction</b>          | Represents a function that accepts an int-valued argument and produces a long-valued result.               |
| <b>IntUnaryOperator</b>           | Represents an operation on a single int-valued operand that produces an int-valued result.                 |
| <b>LongBinaryOperator</b>         | Represents an operation upon two long-valued operands and producing a long-valued result.                  |
| <b>LongConsumer</b>               | Represents an operation that accepts a single long-valued argument and returns no result.                  |
| <b>LongFunction&lt;R&gt;</b>      | Represents a function that accepts a long-valued argument and produces a result.                           |
| <b>LongPredicate</b>              | Represents a predicate (boolean-valued function) of one long-valued argument.                              |
| <b>LongSupplier</b>               | Represents a supplier of long-valued results.                                                              |
| <b>LongToDoubleFunction</b>       | Represents a function that accepts a long-valued argument and produces a double-valued result.             |
| <b>LongToIntFunction</b>          | Represents a function that accepts a long-valued argument and produces an int-valued result.               |
| <b>LongUnaryOperator</b>          | Represents an operation on a single long-valued operand that produces a long-valued result.                |
| <b>ObjDoubleConsumer&lt;T&gt;</b> | Represents an operation that accepts an object-valued and a double-valued argument, and returns no result. |
| <b>ObjIntConsumer&lt;T&gt;</b>    | Represents an operation that accepts an object-valued and a int-valued argument, and returns no result.    |
| <b>ObjLongConsumer&lt;T&gt;</b>   | Represents an operation that accepts an object-valued and a long-valued argument, and returns no result.   |

# java.util.**function**

|                                 |                                                                                                     |
|---------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Predicate</b> <T>            | Represents a predicate (boolean-valued function) of one argument.                                   |
| <b>Supplier</b> <T>             | Represents a supplier of results.                                                                   |
| <b>ToDoubleBiFunction</b> <T,U> | Represents a function that accepts two arguments and produces a double-valued result.               |
| <b>ToDoubleFunction</b> <T>     | Represents a function that produces a double-valued result.                                         |
| <b>ToIntBiFunction</b> <T,U>    | Represents a function that accepts two arguments and produces an int-valued result.                 |
| <b>ToIntFunction</b> <T>        | Represents a function that produces an int-valued result.                                           |
| <b>ToLongBiFunction</b> <T,U>   | Represents a function that accepts two arguments and produces a long-valued result.                 |
| <b>ToLongFunction</b> <T>       | Represents a function that produces a long-valued result.                                           |
| <b>UnaryOperator</b> <T>        | Represents an operation on a single operand that produces a result of the same type as its operand. |



# java.util.**function**

- ▶ Sammlung von Funktionalen Interfaces
- ▶ Definition verschiedener Typen (Namenskonventionen)
  - ▶ **Consumer**: Kein Resultat
  - ▶ **Function**: Produziert Resultat
  - ▶ **Operator**: Produziert Resultat vom Argument-Typ
  - ▶ **Supplier**: Produziert Resultat ohne Argument
  - ▶ **Predicate**: Produziert boolean-Resultat



# IntConsumer

## Method Summary

| All Methods       | Instance Methods   | Abstract Methods                           | Default Methods                                                                                            |
|-------------------|--------------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Modifier and Type |                    | Method and Description                     |                                                                                                            |
| void              |                    | <b>accept</b> (int value)                  | Performs this operation on the given argument.                                                             |
| default           | <b>IntConsumer</b> | <b>andThen</b> ( <b>IntConsumer</b> after) | Returns a composed IntConsumer that performs, in sequence, this operation followed by the after operation. |

```
public static void main(String[] args) {  
    IntConsumer intConsumer = (a) -> System.out.println(a);  
    intConsumer.accept(10);  
}
```

# IntConsumer - andThen

## Method Summary

| All Methods       | Instance Methods   | Abstract Methods                   | Default Methods                                                                                            |
|-------------------|--------------------|------------------------------------|------------------------------------------------------------------------------------------------------------|
| Modifier and Type |                    | Method and Description             |                                                                                                            |
| void              |                    | <b>accept</b> (int value)          | Performs this operation on the given argument.                                                             |
| default           | <b>IntConsumer</b> | <b>andThen</b> (IntConsumer after) | Returns a composed IntConsumer that performs, in sequence, this operation followed by the after operation. |

```
default IntConsumer andThen(IntConsumer after) {  
    return (int t) -> { accept(t); after.accept(t); };  
}
```

Rückgabe eines  
Lambda-Ausdrucks!!

```
IntConsumer adder = (a) -> System.out.println(a + a);  
IntConsumer multiplier = (a) -> System.out.println(a * a);  
adder.andThen(multiplier).accept(20);
```

# IntConsumer - andThen

## Method Summary

| All Methods       | Instance Methods   | Abstract Methods                   | Default Methods                                                                                            |
|-------------------|--------------------|------------------------------------|------------------------------------------------------------------------------------------------------------|
| Modifier and Type |                    | Method and Description             |                                                                                                            |
| void              |                    | <b>accept</b> (int value)          | Performs this operation on the given argument.                                                             |
| default           | <b>IntConsumer</b> | <b>andThen</b> (IntConsumer after) | Returns a composed IntConsumer that performs, in sequence, this operation followed by the after operation. |

```
default IntConsumer andThen(IntConsumer after) {  
    return (int t) -> { accept(t); after.accept(t); };  
}
```

Rückgabe eines  
Lambda-Ausdrucks!!

```
IntConsumer adder = (a) -> System.out.println(a + a);  
IntConsumer multiplier = (a) -> System.out.println(a * a);  
adder.andThen(multiplier).accept(20);
```

40  
400

# Consumers

- ▶ IntConsumer
- ▶ DoubleConsumer
- ▶ Consumer<T>
- ▶ LongConsumer
- ▶ BiConsumer<T,U>
- ▶ ObjDoubleConsumer<T>
- ▶ ObjIntConsumer<T>
- ▶ ObjLongConsumer<T>

# Lambda-Ausdrücke **Beispiel**



► **StudentExample.java**

# java.util.**function**

- ▶ Sammlung von Funktionalen Interfaces
- ▶ Definition verschiedener Typen (Namenskonventionen)
  - ▶ **Consumer**: Kein Resultat
  - ▶ **Function**: Produziert Resultat
  - ▶ **Operator**: Produziert Resultat vom Argument-Typ
  - ▶ **Supplier**: Produziert Resultat ohne Argument
  - ▶ **Predicate**: Produziert boolean-Resultat

# java.util.**function**

- ▶ Sammlung von Funktionalen Interfaces
- ▶ Definition verschiedener Typen (Namenskonventionen)
  - ▶ **Consumer**: Kein Resultat
  - ▶ **Function**: Produziert Resultat
  - ▶ **Operator**: Produziert Resultat vom Argument-Typ
  - ▶ **Supplier**: Produziert Resultat ohne Argument
  - ▶ **Predicate**: Produziert boolean-Resultat



# IntToDoubleFunction

## Method Summary

### All Methods

### Instance Methods

### Abstract Methods

#### Modifier and Type

#### Method and Description

double

**applyAsDouble**(int value)

Applies this function to the given argument.

```
public static void main(String[] args) {  
    IntToDoubleFunction i = (x) -> {return Math.sin(x);};  
    System.out.println(i.applyAsDouble(2));  
}
```

# Functions

- ▶ BiFunction<T,U,R>
- ▶ DoubleFunction<R>
- ▶ Function<T,R>
- ▶ IntFunction<T>
- ▶ IntToDoubleFunction
- ▶ IntToLongFunction
- ▶ LongFunction<R>
- ▶ LongToDoubleFunction
- ▶ LongToIntFunction
- ▶ ToDoubleBiFunction<T,U>
- ▶ ToDoubleFunction<T>
- ▶ ToIntBiFunction<T,U>
- ▶ ToIntFunction<T>
- ▶ ToLongBiFunction<T,U>
- ▶ ToLongFunction<T>

# java.util.**function**

- ▶ Sammlung von Funktionalen Interfaces
- ▶ Definition verschiedener Typen (Namenskonventionen)
  - ▶ **Consumer**: Kein Resultat
  - ▶ **Function**: Produziert Resultat
  - ▶ **Operator**: Produziert Resultat vom Argument-Typ
  - ▶ **Supplier**: Produziert Resultat ohne Argument
  - ▶ **Predicate**: Produziert boolean-Resultat

# IntBinaryOperator

## Method Summary

### All Methods

### Instance Methods

### Abstract Methods

#### Modifier and Type

#### Method and Description

int

**applyAsInt**(int left, int right)

Applies this operator to the given operands.

```
public static void main(String[] args) {  
    IntBinaryOperator io = (x,y)-> x + y;  
    System.out.println(io.applyAsInt(2,3));  
}
```

# Operators

- ▶ BinaryOperator<T>
- ▶ DoubleBinaryOperator
- ▶ DoubleUnaryOperator
- ▶ IntBinaryOperator
- ▶ IntUnaryOperator
- ▶ LongBinaryOperator
- ▶ LongUnaryOperator
- ▶ UnaryOperator<T>

# java.util.**function**

- ▶ Sammlung von Funktionalen Interfaces
- ▶ Definition verschiedener Typen (Namenskonventionen)
  - ▶ **Consumer**: Kein Resultat
  - ▶ **Function**: Produziert Resultat
  - ▶ **Operator**: Produziert Resultat vom Argument-Typ
  - ▶ **Supplier**: Produziert Resultat ohne Argument
  - ▶ **Predicate**: Produziert boolean-Resultat

# BooleanSupplier

## Method Summary

### All Methods

### Instance Methods

### Abstract Methods

#### Modifier and Type

#### Method and Description

boolean

`getAsBoolean()`

Gets a result.

```
BooleanSupplier bs = () -> {  
    LocalDateTime now = LocalDateTime.now();  
    return now.getYear() > 2017;  
};
```

```
BooleanSupplier rbs = () -> {  
    Random ran = new Random();  
    return ran.nextInt(10) < 8;  
};
```



# Suppliers

- ▶ BooleanSupplier
- ▶ DoubleSupplier
- ▶ IntSupplier
- ▶ LongSupplier
- ▶ Supplier<T>

# java.util.**function**

- ▶ Sammlung von Funktionalen Interfaces
- ▶ Definition verschiedener Typen (Namenskonventionen)
  - ▶ **Consumer**: Kein Resultat
  - ▶ **Function**: Produziert Resultat
  - ▶ **Operator**: Produziert Resultat vom Argument-Typ
  - ▶ **Supplier**: Produziert Resultat ohne Argument
  - ▶ **Predicate**: Produziert boolean-Resultat

# IntPredicate

## Method Summary

| All Methods       | Instance Methods | Abstract Methods                | Default Methods                                                                                            |
|-------------------|------------------|---------------------------------|------------------------------------------------------------------------------------------------------------|
| Modifier and Type |                  | Method and Description          |                                                                                                            |
| default           | IntPredicate     | <b>and</b> (IntPredicate other) | Returns a composed predicate that represents a short-circuiting logical AND of this predicate and another. |
| default           | IntPredicate     | <b>negate</b> ()                | Returns a predicate that represents the logical negation of this predicate.                                |
| default           | IntPredicate     | <b>or</b> (IntPredicate other)  | Returns a composed predicate that represents a short-circuiting logical OR of this predicate and another.  |
| boolean           |                  | <b>test</b> (int value)         | Evaluates this predicate on the given argument.                                                            |

```
public static void main(String[] args) {  
    IntPredicate ip = (x) -> x < 0;  
    System.out.println(ip.test(42));  
}
```

false

# IntPredicate - and

```
default IntPredicate and(IntPredicate other) {  
    return (value) -> test(value) && other.test(value);  
}
```

```
IntPredicate intPredGreater = (x) -> x > 10;  
IntPredicate intPredLesser = (x) -> x < 20;  
IntPredicate combine = intPredGreater.and(intPredLesser);
```

```
System.out.println(combine.test(17));  
System.out.println(combine.test(25));
```

false

true

# IntPredicate - **negate**

```
default IntPredicate negate() {  
    return (value) -> !test(value);  
}
```

```
IntPredicate intPredGreater = (x) -> x > 10;  
IntPredicate negation=intPredGreater.negate();  
System.out.println(negation.test(11));  
System.out.println(negation.test(9));
```



# IntPredicate - or

```
default IntPredicate or(IntPredicate other) {  
    return (value) -> test(value) || other.test(value);  
}
```

```
IntPredicate intPred1 = (x) -> x > 10 && x < 20;  
IntPredicate intPred2 = (x) -> x > 40 && x < 50;
```

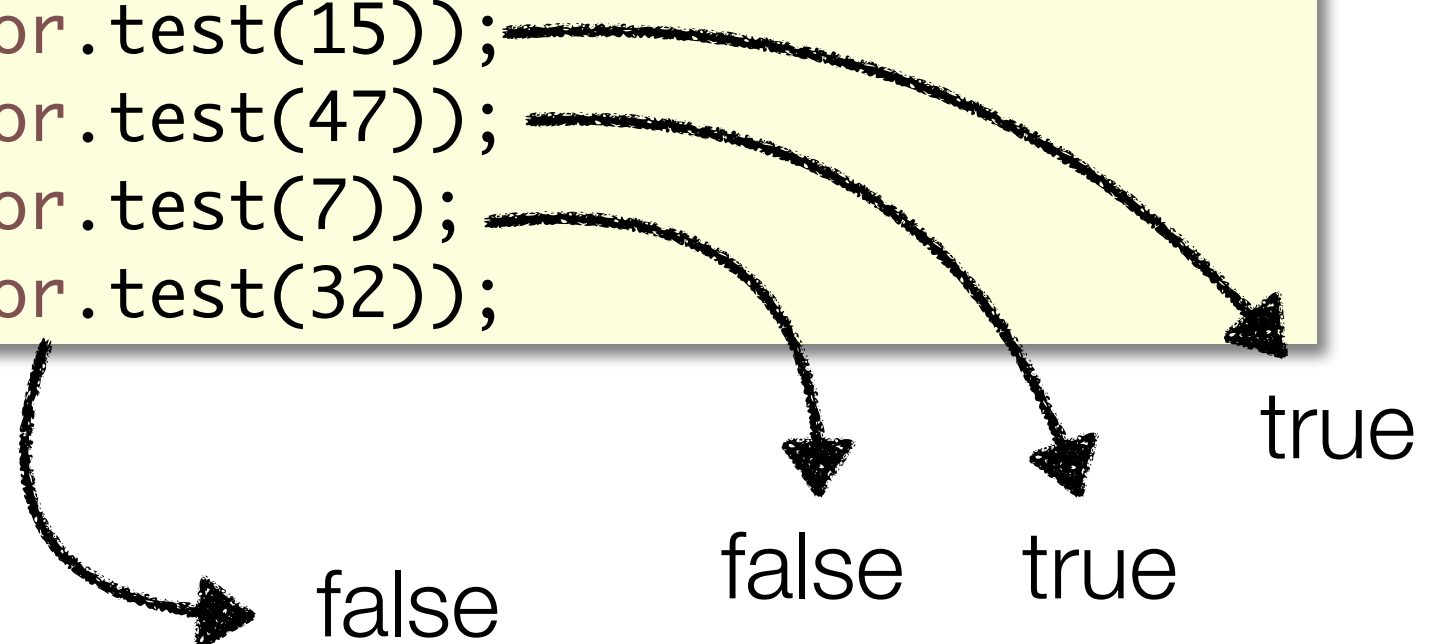
```
IntPredicate or = intPred1.or(intPred2);
```

```
System.out.println(or.test(15));
```

```
System.out.println(or.test(47));
```

```
System.out.println(or.test(7));
```

```
System.out.println(or.test(32));
```



# Predicates

- ▶ DoublePredicate
- ▶ IntPredicate
- ▶ LongPredicate
- ▶ Predicate<T>



# Funktionale Interfaces

Aufgabe: Laden Sie sich das Template `DataSetStructure.java` herunter und erweitern Sie die Klasse so, wie dort angegeben.

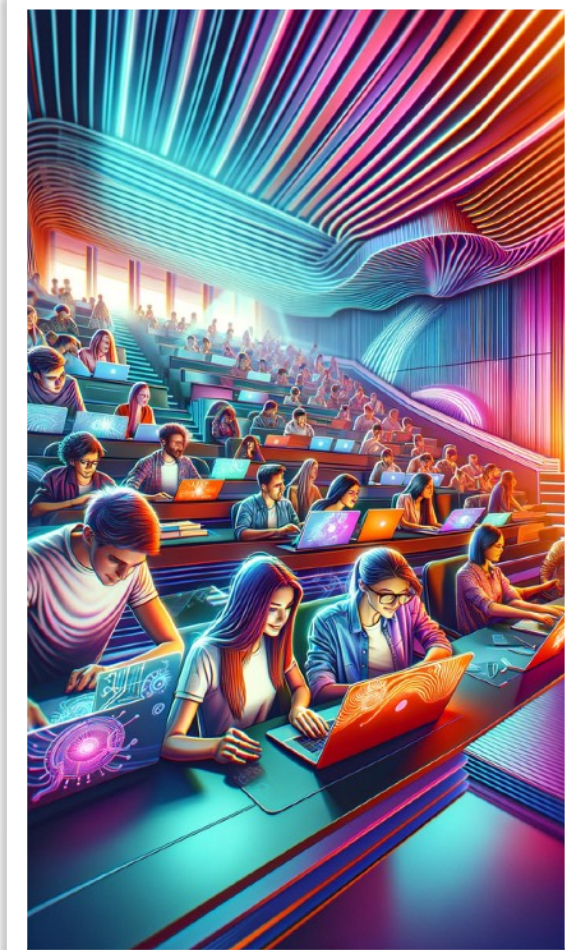
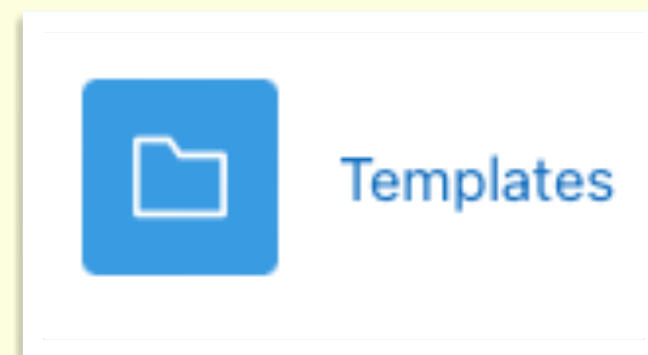
```
public class DataSetStructure {  
    int[] a;  
    public DataSetStructure(int size){  
        a = new int[size];  
        Random ran = new Random();  
        for(int i=0; i<a.length; i++)  
            a[i] = ran.nextInt(100);  
    }  
}
```

//Aufgabe: Verarbeiten der Werte des Arrays mithilfe eines Consumers

```
public void consumeArray( )  
{  
}
```

//Aufgabe: Setzen neuer Werte mithilfe eines Suppliers

```
public void setNewValues( )  
{  
}  
...  
}
```





# Lambda-Ausdrücke **Beispiel**



► **DataStructure.java**

# Funktionale Interfaces

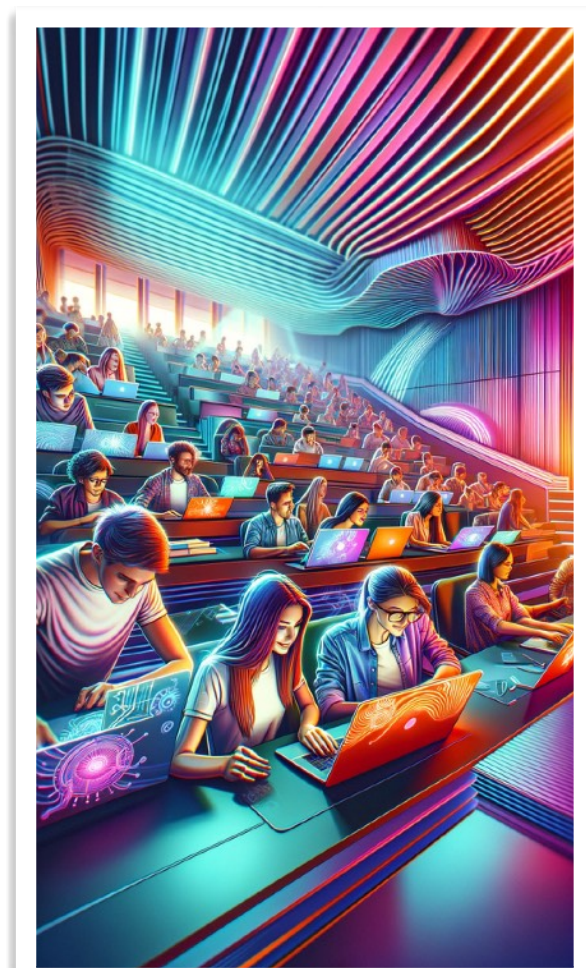
Aufgabe: Laden Sie sich das Template `DataStructureLauncher.java` herunter und erweitern Sie die Klasse so, wie dort angegeben.

```
//Aufgabe: Ausgeben aller Werte größer 10  
IntConsumer greater10 =
```

```
System.out.println("Values > 10:");  
ds.consumeArray(greater10);  
System.out.println("\ndone");
```

```
//Aufgabe: Ausgeben aller Werte kleiner 50  
IntConsumer smaller50 =  
System.out.println("Values < 50:");  
ds.consumeArray(smaller50);  
System.out.println("\ndone");
```

```
//Aufgabe: Ausgeben aller Werte zwischen 10 und 50.  
System.out.println("Values > 10 and Values < 50:");  
ds.consumeArray(    );  
System.out.println("\ndone");  
. . .
```



# Lambda-Ausdrücke **Beispiel**



► **DataStructureLauncher.java**

# Zugriff auf **locale** Variablen

- Lambda-Ausdrücke haben Zugriff auf lokale Variablen (**effectively final**) des umschliessenden Scopes

```
public void doSomething(){  
    final int i = 42;  
    BooleanSupplier bs = () -> {  
        Random ran = new Random();  
        return ran.nextInt(100) < i;  
    };  
}
```



final

# Zugriff auf **locale** Variablen

- Lambda-Ausdrücke haben Zugriff auf lokale Variablen (**effectively final**) des umschliessenden Scopes

```
public void doSomething(){  
    int i = 42;  
    BooleanSupplier bs = () -> {  
        i++;  
        Random ran = new Random();  
        return ran.nextInt(100) < i;  
    };  
}
```

*Local variable *i*  
defined in an  
enclosing scope  
must be final or  
effectively final*



# Zugriff auf **private** Member

- Lambda-Ausdrücke haben Zugriff auf Member der umgebenden Klasse

```
public class SomeClass {  
    private int k = 42;  
    public void doSomething(){  
        BooleanSupplier bs = () -> {  
            return getRandomValue() < k;  
        };  
        bs.getAsBoolean();  
    }  
    private int getRandomValue(){  
        return new Random().nextInt();  
    }  
}
```





# Lambdas: Shadowing

```
public class Shadowing {  
    private int i = 0;  
    class InnerClass{  
        int i = 1;  
        public void someMethod(){  
            IntConsumer lambda = i -> {  
                System.out.println("i: "+i);  
                System.out.println("i: "+this.i);  
                System.out.println("i: "+Shadowing.this.i);  
            };  
            lambda.accept(2);  
        }  
    }  
    public static void main(String[] args){  
        new Shadowing().new InnerClass().someMethod();  
    }  
}
```

2

1

0

# Typdeduktion (**Target Typing**)

- Woher kennt der Compiler Parameter- und Return-Typ?

```
IntPredicate ip = (x) -> x < 0;
```



Zieltyp

```
public interface IntPredicate {  
    boolean test(int value);  
}
```



# Typdeduktion (**Target Typing**)

- Woher kennt der Compiler Parameter- und Return-Typ?

```
IntPredicate ip = (x) -> x < 0;
```

 Zieltyp

```
public interface IntPredicate {  
    boolean test(int value);  
}
```

# Typdeduktion (**Target Typing**)

- Woher kennt der Compiler Parameter- und Return-Typ?

```
BiPredicate<String,String> sp1      = (s,t) -> s.equalsIgnoreCase(t);  
BiFunction<String,String,Boolean> sp2 = (s,t) -> s.equalsIgnoreCase(t) ;
```

```
public interface BiPredicate<T, U> {  
    boolean test(T t, U u);  
}  
public interface BiFunction<T, U, R> {  
    R apply(T t, U u);  
}
```

# Deduktions**kontext**

- ▶ Compiler benötigt Deduktionskontext zum Auflösen
  - ▶ Lambda-Ausdrücke nur an folgenden Stellen zulässig
    - ▶ auf der rechten Seite von Zuweisungen

```
BiPredicate<String,String> sp1      = (s,t) -> s.equalsIgnoreCase(t);  
BiFunction<String,String,Boolean> sp2 = (s,t) -> s.equalsIgnoreCase(t) ;
```

# Deduktions**kontext**

- ▶ Compiler benötigt Deduktionskontext zum Auflösen
  - ▶ Lambda-Ausdrücke nur an folgenden Stellen zulässig
    - ▶ auf der rechten Seite von Zuweisungen
    - ▶ als Argumente in einem Methoden-Aufruf

```
public void applyOperatorToArray(IntUnaryOperator o){...}
```

Zieltyp

```
applyOperatorToArray(x -> x*2);
```

# Deduktions**kontext**

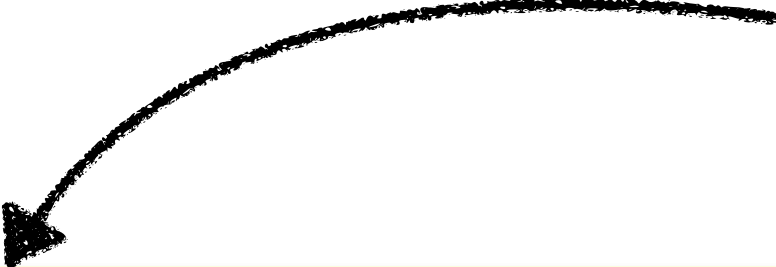
- ▶ Compiler benötigt Deduktionskontext zum Auflösen
  - ▶ Lambda-Ausdrücke nur an folgenden Stellen zulässig
    - ▶ auf der rechten Seite von Zuweisungen
    - ▶ als Argumente in einem Methoden-Aufruf
    - ▶ als Returnwert in einer return-Anweisung



```
default IntConsumer andThen(IntConsumer after) {  
    return (int t) -> { accept(t); after.accept(t); };  
}
```

# Deduktions**kontext**

- ▶ Compiler benötigt Deduktionskontext zum Auflösen
  - ▶ Lambda-Ausdrücke nur an folgenden Stellen zulässig
    - ▶ auf der rechten Seite von Zuweisungen
    - ▶ als Argumente in einem Methoden-Aufruf
    - ▶ als Returnwert in einer return-Anweisung
    - ▶ in einem Cast-Ausdruck

 Zieltyp

```
Object u = (IntUnaryOperator) x -> x*2;
```

# Typdeduktion

```
Object o = (s,t) -> s.equalsIgnoreCase(t) ;
```



The target type  
of this  
expression must  
be a functional  
interface

```
Object o = (BiPredicate<String,String>) (s,t)  
-> s.equalsIgnoreCase(t);
```





```
public interface Runnable {  
    void run();  
}  
public interface Callable<V> {  
    V call();  
}
```



60 Sekunden

```
public class TargetTypeExample {  
    void invoke(Runnable r) {  
        r.run();  
    }  
    <T> T invoke(Callable<T> c) throws Exception {  
        return c.call();  
    }  
    public static void main(String[] args) throws Exception {  
        TargetTypeExample tte = new TargetTypeExample();  
        String s = tte.invoke(() -> "done");  
    }  
}
```

Welchen Typ  
hat der  
Lambda-  
Ausdruck?



```
public interface Runnable {  
    void run();  
}  
public interface Callable<V> {  
    V call();  
}
```

Antwort:  
Callable<String>

```
public class TargetTypeExample {  
    void invoke(Runnable r) {  
        r.run();  
    }  
    <T> T invoke(Callable<T> c) throws Exception {  
        return c.call();  
    }  
    public static void main(String[] args) throws Exception {  
        TargetTypeExample tte = new TargetTypeExample();  
        String s = tte.invoke(() -> "done");  
    }  
}
```

Welchen Typ  
hat der  
Lambda-  
Ausdruck?

# Modifizierende Lambda-Ausdrücke

- Paradigma funktionaler Programmierung:  
Funktionen ändern keine Daten sondern liefern nur ein Ergebnis und sind idempotent

Setzen die Lambda-Ausdrücke in Java dieses Paradigma um?



# Modifizierende Lambda-Ausdrücke

- Paradigma funktionaler Programmierung:  
Funktionen ändern keine Daten sondern liefern nur ein Ergebnis und sind idempotent

Setzen die Lambda-Ausdrücke in Java dieses Paradigma um?



# Modifizierende Lambda-Ausdrücke

```
IntPredicate isEven = (int i) -> { return i%2==0; };
```

nicht  
modifizierend  
(und idempotent)



```
List<Point> points = new ArrayList<>();  
...  
points.forEach(p -> { p.x = 0; });
```

modifizierend!  
(aber idempotent)



forEach erwartet  
einen Consumer  
als Argument



# Modifizierende Lambda-Ausdrücke

```
IntPredicate isEven = (int i) -> { return i%2==0; };
```

nicht  
modifizierend  
(und idempotent)



```
List<Point> points = new ArrayList<>();  
...  
points.forEach(p -> { p.x += 1; });
```

forEach erwartet  
einen Consumer  
als Argument



modifizierend!  
(nicht idempotent!)





# Modifizierende Lambda-Ausdrücke



Concurrent  
Modification  
Exception!!!

```
List<Point> points = new ArrayList<>();  
...  
points.forEach(p -> points.add(new Point(0,p.y)));
```

# Methoden-Referenzen

- ▶ Lambda-Ausdrücke ermöglichen die Übergabe von Funktionen(Methoden) als Argumente
  - ▶ einfacher als Callback-Konstrukte
  - ▶ manchmal aber immer noch zu umständlich
  - ▶ Beispiel:

```
List<Integer> numbers = new ArrayList<>();  
.  
.  
.  
numbers.forEach( i -> System.out.println(i) );
```

Lambda-Ausdruck  
ruft lediglich eine  
Methode auf



# Methoden-Referenzen

```
List<Integer> numbers = new ArrayList<>();  
...  
numbers.forEach( i -> System.out.println(i) );
```

Lambda-Ausdruck  
ruft lediglich eine  
Methode auf

einfacher:  
Methoden-Referenzen

```
List<Integer> numbers = new ArrayList<>();  
...  
numbers.forEach( System.out :: println );
```



# Methoden-Referenzen

```
List<Integer> numbers = new ArrayList<>();  
...  
numbers.forEach( i -> System.out.println(i) );
```

Lambda-Ausdruck  
ruft lediglich eine  
Methode auf

einfacher:  
Methoden-Referenzen

```
List<Integer> numbers = new ArrayList<>();  
...  
numbers.forEach( System.out :: println );
```

Receiver  
(Object oder Klasse)

Methodenname  
(ohne Argumente)

# Methoden-**Referenzen**

Was geschieht mit den  
**Methodenargumenten ?**

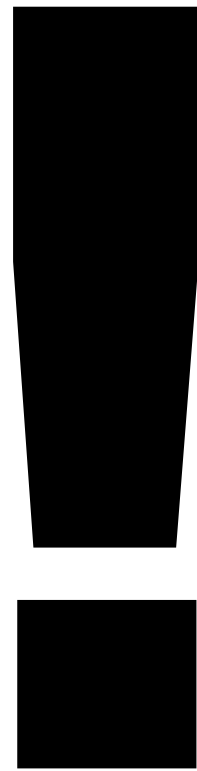
Warum ist das ein  
gültiger **Ausdruck ?**

Wie löst der **Compiler**  
diesen Ausdruck auf ?



# Methoden-Referenzen

- ▶ Methoden Referenzen können ausschließlich zum Ersetzen eines Lambda-Ausdrucks mit nur einem Methoden-Aufruf genutzt werden



# Methoden-**Referenzen**

Anonymous Class



if(implementiert funktionales Interface)

ersetzbar durch:

Lambda-Ausdruck



if(nur ein Methodenaufruf)

ersetzbar durch:

Methoden-Referenz





# Methoden-Referenzen **Beispiel**



► **Roster.java**

# Methoden-Referenzen

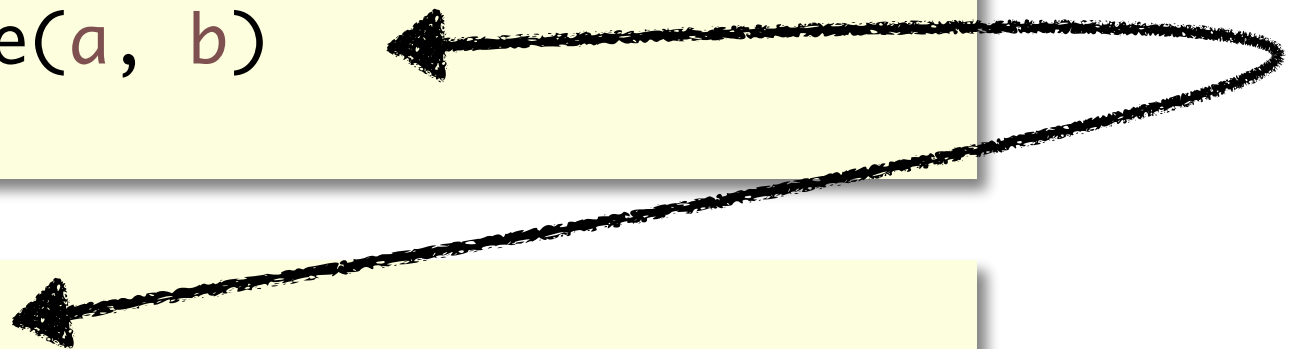
```
Roster r = new Roster();  
r.sort(new PersonComparator() {  
    public int compare(Person a, Person b) {  
        return a.getBirthdate().compareTo(b.getBirthdate());  
    }  
});
```

```
r.sort((Person a, Person b) -> {  
    return a.getBirthdate().compareTo(b.getBirthdate());  
});
```

```
r.sort(  
    (a, b) -> Person.compareByAge(a, b)  
);
```

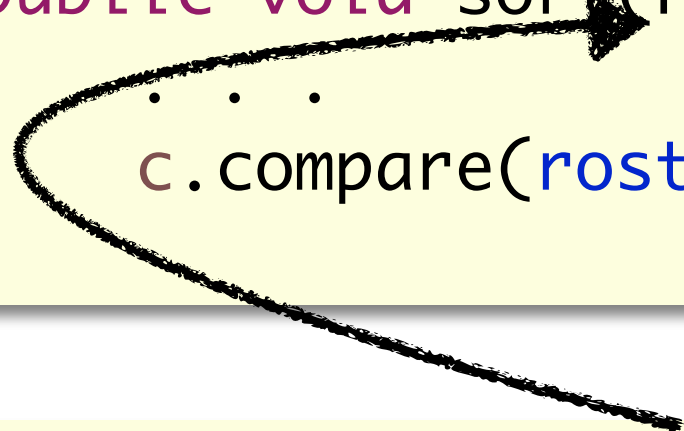
```
r.sort(Person::compareByAge);
```

semantisch  
identisch



# Typdeduktion

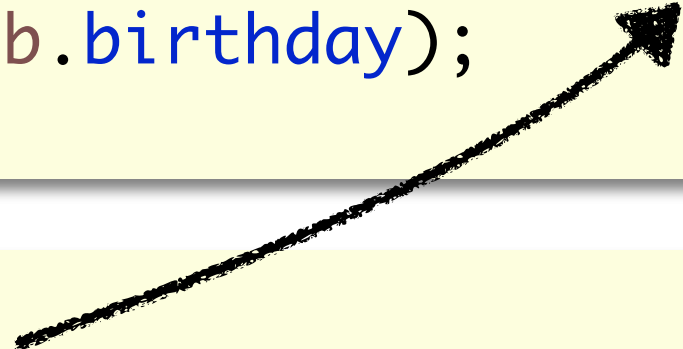
```
public class Roster {  
    public void sort(PersonComparator c){  
        .  
        .  
        .  
        c.compare(roster.get(j), roster.get(j+1)) }  
}
```



```
r.sort(Person::compareByAge);
```



```
public static int compareByAge(Person a, Person b) {  
    return a.birthday.compareTo(b.birthday);  
}
```



```
interface PersonComparator{  
    int compare(Person a, Person b);  
}
```

# Typdeduktion

```
public class Roster {  
    public void sort(PersonComparator c){  
        . . .  
        c.compare(roster.get(j), roster.get(j+1)) }  
    }
```

A curved arrow originates from the `sort` method call in the second code block and points to the `sort` method definition in this first code block.

```
r.sort(Person::compareByAge);
```

```
public static int compareByAge(Person a, Person b) {  
    return a.birthday.compareTo(b.birthday);  
}
```

A straight arrow originates from the `compareByAge` method call in the second code block and points to the `compareByAge` method definition in this third code block.

```
interface PersonComparator{  
    int compare(Person a, Person b);  
}
```

A straight arrow originates from the `compare` method call in the third code block and points to the `compare` method definition in this fourth code block.

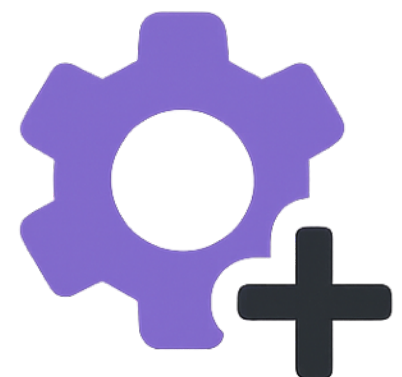


# Deduktions**kontext**

- ▶ Compiler benötigt Deduktionskontext zum Auflösen
  - ▶ Methoden-Referenzen nur an folgenden Stellen zulässig
    - ▶ auf der rechten Seite von Zuweisungen
    - ▶ als Argumente in einem Methoden-Aufruf
    - ▶ als Returnwert in einer return-Anweisung
    - ▶ in einem Cast-Ausdruck

# Methoden-Referenz **Typen**

- ▶ Referenzen zu **statischen** Methoden
- ▶ Referenzen zu Instanz-Methoden eines gegebenen **Typs**
- ▶ Referenzen zu Instanz-Methoden eines **Objekts**
- ▶ Referenzen zu **Konstruktoren**



# Statische Methoden-Referenzen

- ▶ Haben wir einen Lambda-Ausdruck folgender Form:

```
(args) -> Class.staticMethod(args)
```

- ▶ Dann können wir folgendes schreiben:

```
Class::staticMethod
```

# Statische Methoden-Referenzen

```
class Numbers {  
    public static boolean isMoreThanFifty(int n1, int n2) {  
        return (n1 + n2) > 50;  
    }  
    public static List<Integer> findNumbers(  
        List<Integer> l,  
        BiPredicate<Integer, Integer> p) {  
        List<Integer> newList = new ArrayList<>();  
        for(Integer i : l) {  
            if(p.test(i, i + 10)) {  
                newList.add(i);  
            }  
        }  
        return newList;  
    }  
}
```

# Statische Methoden-Referenzen

```
// Using an anonymous class
```

```
findNumbers(list, new BiPredicate<Integer, Integer>() {  
    public boolean test(Integer i1, Integer i2) {  
        return Numbers.isMoreThanFifty(i1, i2);  
    }  
});
```

```
// Using a lambda expression
```

```
findNumbers(list, (i1, i2) -> Numbers.isMoreThanFifty(i1, i2));
```

```
// Using a method reference
```

```
findNumbers(list, Numbers::isMoreThanFifty);
```

# Statische Methoden-Referenzen

```
class Numbers {  
    public static boolean isMoreThanFifty(int n1, int n2) {  
        return (n1 + n2) > 50;  
    }  
    public static List<Integer> findNumbers(  
        List<Integer> l,  
        BiPredicate<Integer, Integer> p) {  
        List<Integer> newList = new ArrayList<>();  
        for(Integer i : l) {  
            if(p.test(i, i + 10)) {  
                newList.add(i);  
            }  
        }  
        return newList;  
    }  
}
```

```
findNumbers(list, Numbers::isMoreThanFifty);
```

# Statische Methoden-Referenzen

```
class Numbers {  
    public static boolean isMoreThanFifty(int n1, int n2) {  
        return (n1 + n2) > 50;  
    }  
    public static List<Integer> findNumbers(  
        List<Integer> l,  
        BiPredicate<Integer, Integer> p) {  
        List<Integer> newList = new ArrayList<>();  
        for(Integer i : l) {  
            if(p.test(i, i + 10)) {  
                newList.add(i);  
            }  
        }  
        return newList;  
    }  
}
```

```
public boolean test(Integer i1, Integer i2){  
    return Numbers.isMoreThanFifty(i1, i2);  
}
```

```
findNumbers(list, Numbers::isMoreThanFifty);
```



# Statische Methoden-Referenzen

# Zieltyp

```
BiPredicate<Integer, Integer>
```

```
p.test(i, i + 10)
```

```
public boolean test(Integer i1, Integer i2){  
  
}  

```

# Statische Methoden-Referenzen

Zieltyp

BiPredicate<Integer, Integer>

p.test(i, i + 10)

```
public boolean test(Integer i1, Integer i2){  
    return Numbers.isMoreThanFifty(i1, i2);  
}
```

findNumbers(list, Numbers::isMoreThanFifty);

# Statische Methoden-Referenzen

Zieltyp

BiPredicate<Integer, Integer>

p.test(i, i + 10)

```
public boolean test(Integer i1, Integer i2){  
    return Numbers.isMoreThanFifty(i1, i2);  
}
```

findNumbers(list, Numbers::isMoreThanFifty);

```
public static boolean isMoreThanFifty(int n1, int n2) {  
    return (n1 + n2) > 50;  
}
```

# Methoden-Referenzen **Beispiel**



► **Numbers.java**

# Statische Methoden-Referenzen

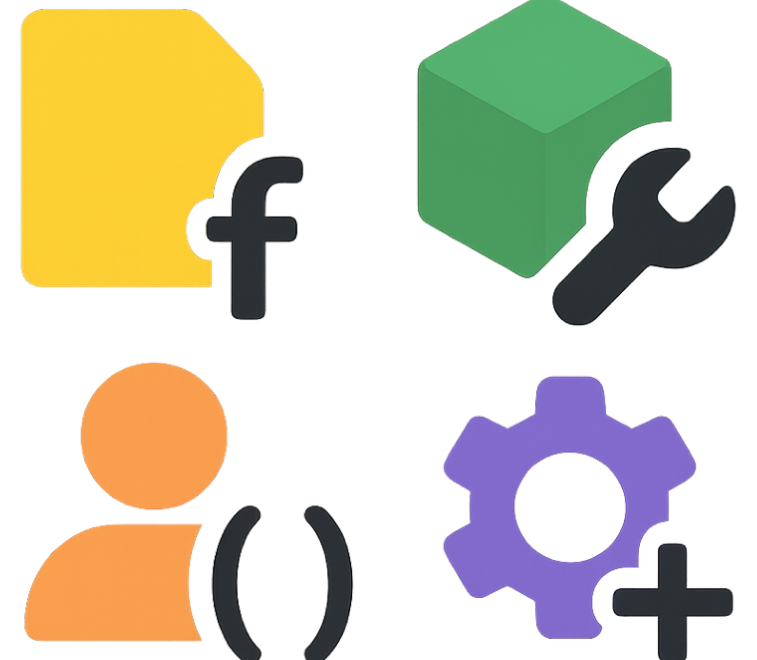
```
class Numbers {  
    public static boolean isMoreThanFifty(int n1, int n2) {  
        return (n1 + n2) > 50;  
    }  
    public static boolean isMoreThanFifty(int n1, int n2, int n3){  
        return (n1 + n2 + n3) > 50;  
    }  
    public static List<Integer> findNumbers(  
        List<Integer> l, BiPredicate<Integer, Integer> p) {. . .}  
  
    public static List<Integer> findNumbers(  
        List<Integer> l, TriFunction p) {. . .}  
}
```

```
findNumbers(list, Numbers::isMoreThanFifty);
```

*Ambiguous  
method  
reference !!!*

# Methoden-Referenz **Typen**

- ▶ Referenzen zu **statischen** Methoden 
- ▶ Referenzen zu Instanz-Methoden eines gegebenen **Typs**
- ▶ Referenzen zu Instanz-Methoden eines **Objekts**
- ▶ Referenzen zu **Konstruktoren**



# Typ Methoden-Referenzen

► Referenzen zu Instanz-Methoden eines gegebenen **Typs**

► Haben wir einen Lambda-Ausdruck folgender Form:

```
(obj, args) -> obj.instanceMethod(args)
```

► Dann können wir folgendes schreiben:

```
ObjectType::instanceMethod
```

Typ nicht Instanz!





# Typ Methoden-Referenzen

```
class Shipment {  
    public double calculateWeight() {  
        double weight = 0;  
        // Calculate weight  
        return weight;  
    }  
    public List<Double> calculateOnShipments(  
        List<Shipment> l, Function<Shipment, Double> f)  
    {  
        List<Double> results = new ArrayList<>();  
        for(Shipment s : l) {  
            results.add(f.apply(s));  
        }  
        return results;  
    }  
}
```



# Typ Methoden-Referenzen

```
// Using an anonymous class
calculateOnShipments(l, new Function<Shipment, Double>() {
    public Double apply(Shipment s) { // The object
        return s.calculateWeight(); // The method
    }
});
```

```
// Using a lambda expression
calculateOnShipments(l, s -> s.calculateWeight());
```

```
// Using a method reference
calculateOnShipments(l, Shipment::calculateWeight);
```

# Typ Methoden-Referenzen

```
class Shipment {  
    public double calculateWeight() {  
        double weight = 0;  
        // Calculate weight  
        return weight;  
    }
```

Bildung einer  
Referenz zur  
Instanz-Methode

```
    public List<Double> calculateOnShipments(  
        List<Shipment> l, Function<Shipment, Double> f)  
    {  
        List<Double> results = new ArrayList<>();  
        for(Shipment s : l) {  
            results.add(f.apply(s));  
        }  
        return results;  
    }  
}
```

Referenz zur  
Methode der  
Klasse

```
calculateOnShipments(l, Shipment::calculateWeight);
```

# Typ Methoden-Referenzen

```
class Shipment {  
    public double calculateWeight() {  
        double weight = 0;  
        // Calculate weight  
        return weight;  
    }
```

Bildung einer  
Referenz zur  
Instanz-Methode

```
    public List<Double> calculateOnShipments(  
        List<Shipment> l, Function<Shipment, Double> f)  
    {  
        List<Double> results = new ArrayList<>();  
        for(Shipment s : l) {  
            results.add(f.apply(s));  
        }  
        return results;  
    }  
}
```

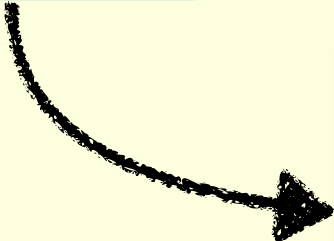
Referenz zur  
Methode der  
Klasse

Typ muss  
passen!

```
calculateOnShipments(l, Shipment::calculateWeight);
```

# Typ Methoden-Referenzen

```
class Shipment {  
    public double calculateWeight() {  
        double weight = 0;  
        // Calculate weight  
        return weight;  
    }  
    public List<Double> calculateOnShipments(  
        List<Shipment> l, Function<Shipment, Double> f)  
    {  
        List<Double> results = new ArrayList<>();  
        for(Shipment s : l) {  
            results.add(f.apply(s));  
        }  
        return results;  
    }  
}
```



```
public Double apply(Shipment s){  
    return s.calculateWeight();  
}
```

```
calculateOnShipments(l, Shipment::calculateWeight);
```

# Methoden-Referenzen **Beispiel**



► **Shipment.java**

# Methoden-Referenzen Beispiel

```
public double calculateWeight() {  
    double weight = 0;  
    // Calculate weight  
    return weight;  
}  
public static double calculateWeight(Shipment s)  
{  
    double weight = 42;  
    // Calculate weight  
    return weight;  
}
```

Ambiguous  
method  
reference !!!

```
public Double apply(Shipment s){  
    return s.calculateWeight();}
```

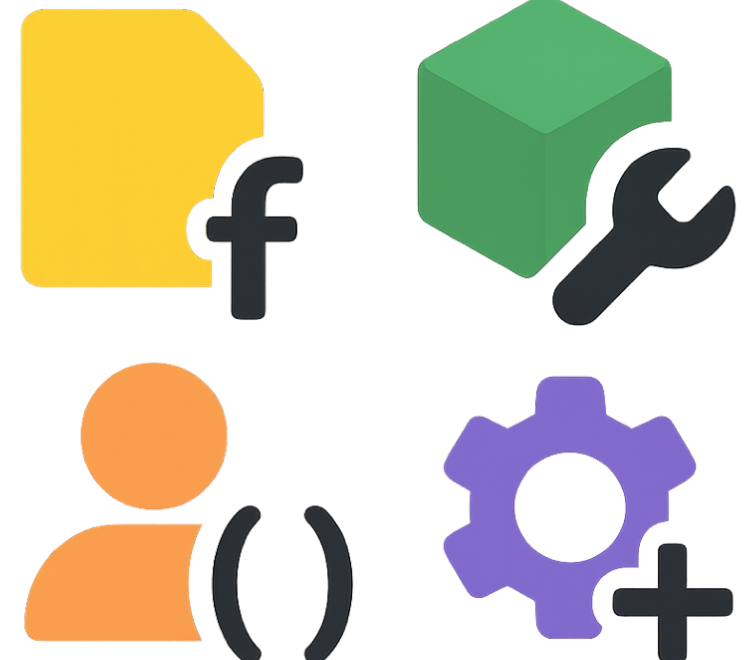
```
public Double apply(Shipment s){  
    return Shipment.calculateWeight(s);}
```

```
calculateOnShipments(l, Shipment::calculateWeight);
```



# Methoden-Referenz **Typen**

- ▶ Referenzen zu **statischen** Methoden 
- ▶ Referenzen zu Instanz-Methoden eines gegebenen **Typs** 
- ▶ Referenzen zu Instanz-Methoden eines **Objekts**
- ▶ Referenzen zu **Konstruktoren**





# Objekt Methoden-Referenzen

- ▶ Referenzen zu Instanz-Methoden eines **Objekts**
- ▶ Haben wir einen Lambda-Ausdruck folgender Form:

```
(args) -> obj.instanceMethod(args)
```

- ▶ Dann können wir folgendes schreiben:

```
obj::instanceMethod
```

Außerhalb des  
Lambda-Ausdrucks  
instanciiertes  
Objekt

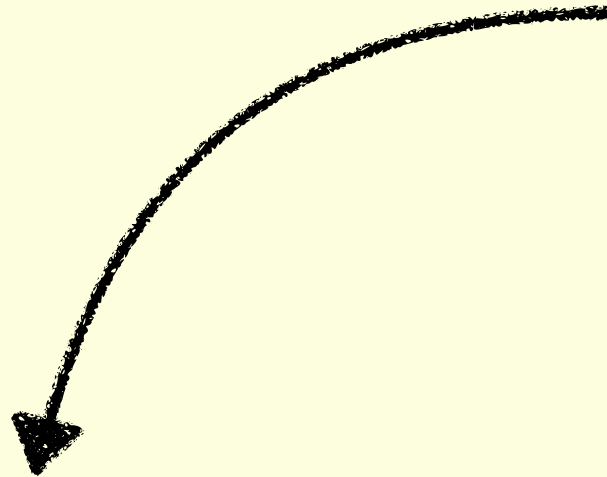




# Objekt Methoden-Referenzen

```
class Car {  
    private int id;  
    private String color;  
    public int getId() {  
        return id;  
    }  
}
```

Funktionales  
Interface



```
    public void execute(Consumer<Car> c) {  
        c.accept(this);  
    }  
}
```

```
class Mechanic {  
    public void fix(Car c) {  
        System.out.println("Fixing car " + c.getId());  
    }  
}
```

# Objekt Methoden-Referenzen

```
final Mechanic mechanic = new Mechanic();  
Car car = new Car();
```

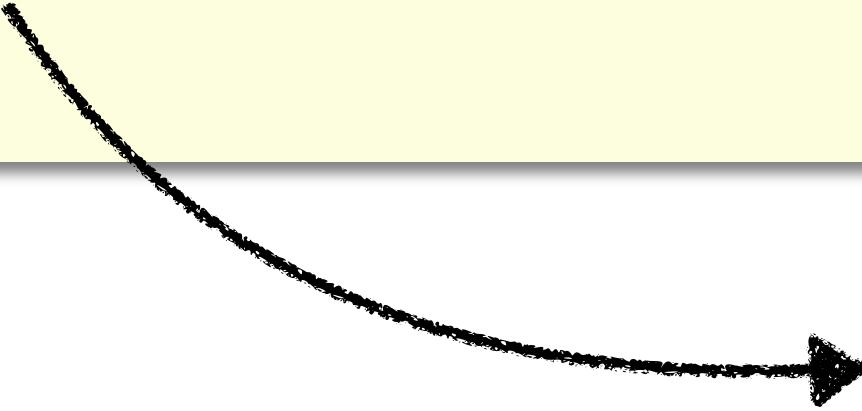
```
// Using an anonymous class  
car.execute(new Consumer<Car>() {  
    public void accept(Car c) {  
        mechanic.fix(c);  
    }  
});
```

```
// Using a lambda expression  
car.execute(c -> mechanic.fix(c));
```

```
// Using a method reference  
car.execute(mechanic::fix);
```

# Objekt Methoden-Referenzen

```
class Car {  
    private int id;  
    private String color;  
    public int getId() {  
        return id;  
    }  
  
    public void execute(Consumer<Car> c) {  
        c.accept(this);  
    }  
}
```



```
public void accept(Car c) {  
    mechanic.fix(c);  
}
```

```
car.execute(mechanic::fix);
```

# Objekt Methoden-Referenzen

```
class Car {  
    private int id;  
    private String color;  
    public int getId() {  
        return id;  
    }  
  
    public void execute(Consumer<Car> c) {  
        c.accept(this);  
    }  
}
```

`car.execute(mechanic::fix);`

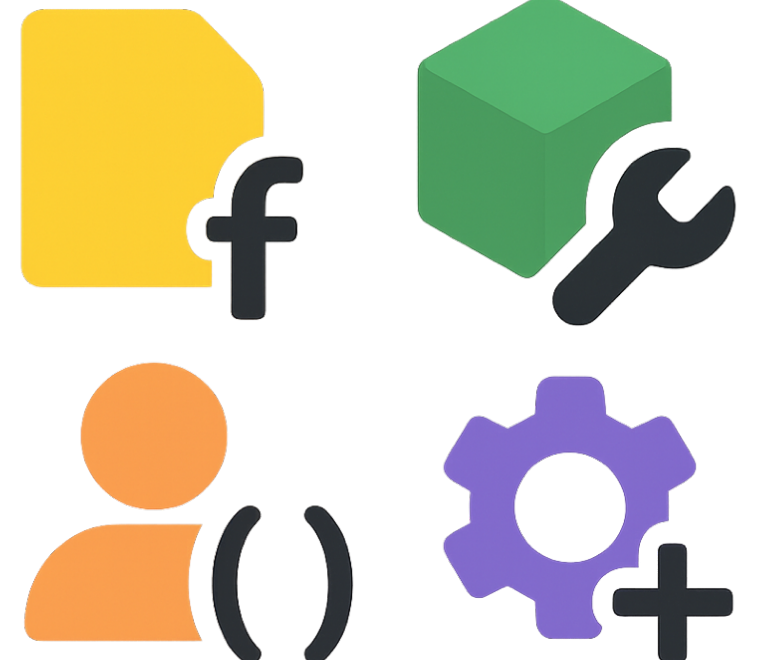
**≠**

`car.execute(Mechanic::fix);`

Achtung Object vs. Typ

# Methoden-Referenz **Typen**

- ▶ Referenzen zu **statischen** Methoden 
- ▶ Referenzen zu Instanz-Methoden eines gegebenen **Typs** 
- ▶ Referenzen zu Instanz-Methoden eines **Objekts** 
- ▶ Referenzen zu **Konstruktoren**



# Konstruktor-Referenzen

- ▶ Referenzen zu **Konstruktor**en

- ▶ Haben wir einen Lambda-Ausdruck folgender Form:

```
(args) -> new ClassName(args)
```

- ▶ Dann können wir folgendes schreiben:

```
ClassName::new
```

Lambda-Ausdruck  
instanziert lediglich  
ein neues Objekt



# Konstruktor-Referenzen

```
// Using an anonymous class
```

```
Supplier<List<String>> s = new Supplier() {  
    public List<String> get() {  
        return new ArrayList<String>();  
    }  
};  
List<String> l = s.get();
```

```
// Using a lambda expression
```

```
Supplier<List<String>> s1 = () -> new ArrayList<String>();  
List<String> l1 = s1.get();
```

```
// Using a method reference
```

```
Supplier<List<String>> s2 = ArrayList<String>::new;  
List<String> l2 = s2.get();
```



# Konstruktor-Referenzen

```
// Using an anonymous class
Function<String, Integer> f1 =
    new Function<String, Integer>() {
        public Integer apply(String s) {
            return new Integer(s);
        }
    };
Integer i1 = f1.apply("100");
```

Konstruktor mit  
einem Argument



```
// Using a lambda expression
Function<String, Integer> f2 = s2 -> new Integer(s2);
Integer i2 = f2.apply("100");
```

```
// Using a method reference
Function<String, Integer> f3 = Integer::new;
Integer i3 = f3.apply("100");
```



# Konstruktor-Referenzen

```
IntUnaryOperator f4 = Integer::new;  
Integer i4 = f4.applyAsInt(100);
```

## Constructor Summary

### Constructors

#### Constructor and Description

**Integer**(int value)

Constructs a newly allocated `Integer` object that represents the specified `int` value.

**Integer**(String s)

Constructs a newly allocated `Integer` object that represents the `int` value indicated by the `String` parameter.

Deduction des  
korrekten  
Konstruktors

```
Function<String, Integer> f3 = Integer::new;  
Integer i3 = f3.apply("100");
```

# Methoden-Referenz **Typen**

- ▶ Referenzen zu **statischen** Methoden 
- ▶ Referenzen zu Instanz-Methoden eines gegebenen **Typs** 
- ▶ Referenzen zu Instanz-Methoden eines **Objekts** 
- ▶ Referenzen zu **Konstruktoren** 



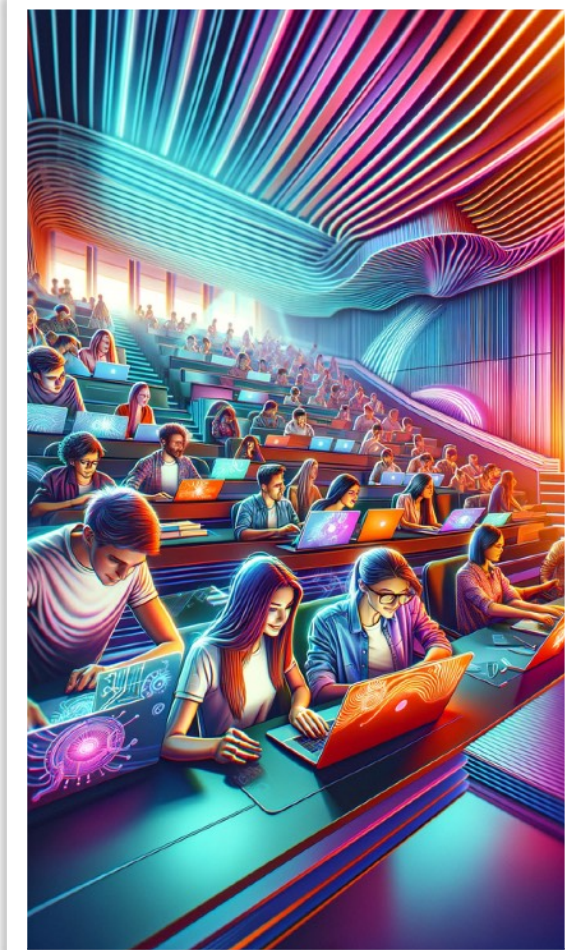
# Methodenreferenzen Übung

Aufgabe: Laden Sie sich das Template MethodenreferenzenUebung.java herunter und erweitern Sie die Klasse so, wie dort angegeben.

```
public static void main(String[] args) {  
    Person person = new Person("Alice", 25);  
  
    // 1. Rufen Sie die Methode performOperationOnStr  
    // Methodenreferenz auf.  
    // Die referenzierte Methode soll die Länge einer String zurückgeben.  
  
    // 2. Rufen Sie die Methode performOperationOnPerson mit einer Objekt-  
    // Methodenreferenz auf, die auf  
    // die getName-Methode des person-Objekts verweist.  
  
    // 3. Rufen Sie die Methode performOperationOnPersonAndAge mit einer Typ-  
    // Methodenreferenz auf, die auf  
    // die setAge-Methode der Person-Klasse verweist.  
  
    // 4. Rufen Sie die Methode createNewPerson mit einer Methodenreferenz auf,  
    // die auf den Konstruktor der Person-Klasse verweist.  
    Person newPerson  
    System.out.println("Kopierte Person: " + newPerson);  
}
```



Templates



# Java vs. funktionale Sprachen

- ▶ In funktionalen Programmiersprachen modifizieren Funktionen keine Daten
  - ▶ Lambda-Ausdrücke in Java können Variablen modifizieren
- ▶ Funktionen in funktionalen Programmiersprachen sind idempotent
  - ▶ Lambda-Ausdrücke nicht zwangsläufig
- ▶ In funktionalen Sprachen gibt es eigene Funktionstypen
  - ▶ Lambda-Ausdrücke in Java als Objekte von funktionalen Interfaces
- ▶ Beispiele funktionaler Sprachen
  - ▶ Lisp, Haskell, SML
  - ▶ (Scala, Swift)

# Was haben Sie **gelernt**?

Erklären Sie Ihrem Sitznachbarn (45 Sekunden):

- Was ist ein Lambda-Ausdruck und wann verwenden wir diese?
- Wie werden Lambda-Ausdrücke in Java umgesetzt?
- Was ist ein Funktionales Interface und warum benötigen wir es?
- Was ist Typ-Deduktion und wo wird es verwendet?
- Was ist eine Methodenreferenz und wo können wir es einsetzen?

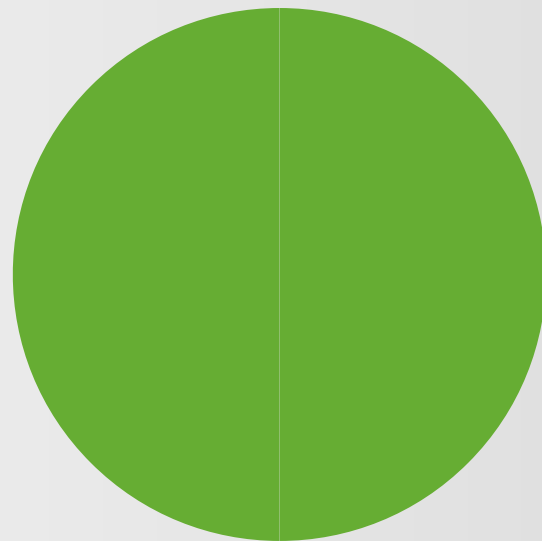


45 Sekunden

# Was haben Sie **gelernt**?

Erklären Sie Ihrem Sitznachbarn (45 Sekunden):

- Was ist ein Lambda-Ausdruck und wann verwenden wir diese?
- Wie werden Lambda-Ausdrücke in Java umgesetzt?
- Was ist ein Funktionales Interface und warum benötigen wir es?
- Was ist Typ-Deduktion und wo wird es verwendet?
- Was ist eine Methodenreferenz und wo können wir es einsetzen?



45 Sekunden

# Was haben Sie **gelernt**?

Erklären Sie Ihrem Sitznachbarn (45 Sekunden):

- Was ist ein Lambda-Ausdruck und wann verwenden wir diese?
- Wie werden Lambda-Ausdrücke in Java umgesetzt?
- Was ist ein Funktionales Interface und warum benötigen wir es?
- Was ist Typ-Deduktion und wo wird es verwendet?
- Was ist eine Methodenreferenz und wo können wir es einsetzen?



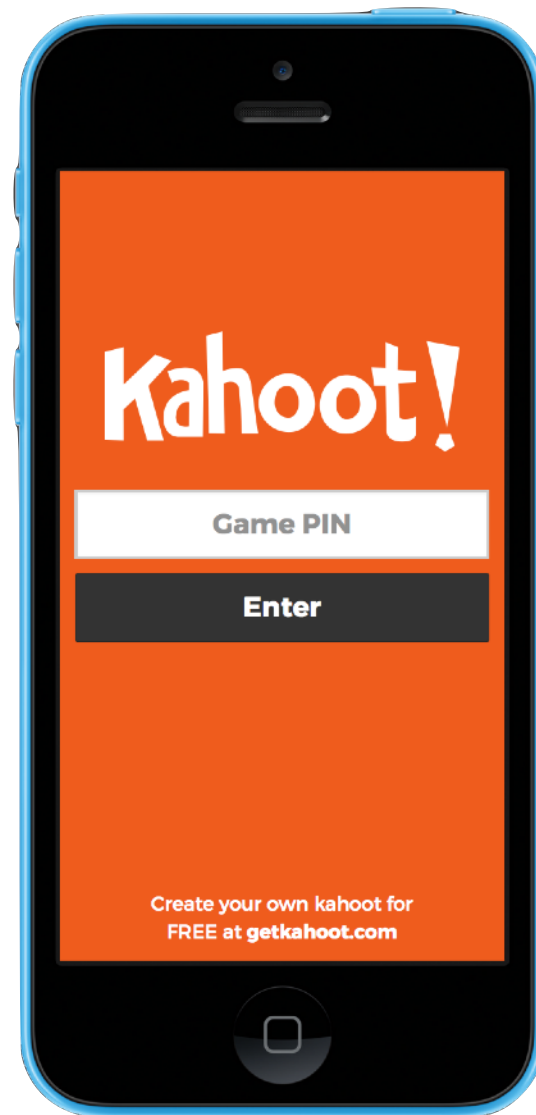
45 Sekunden



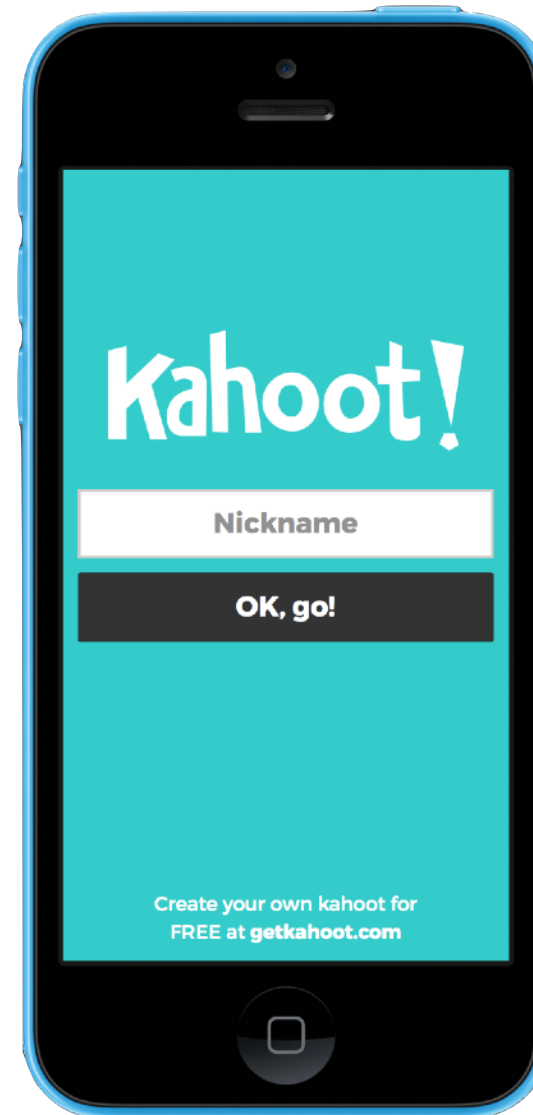
# Kahoot!

go to  
kahoot.it

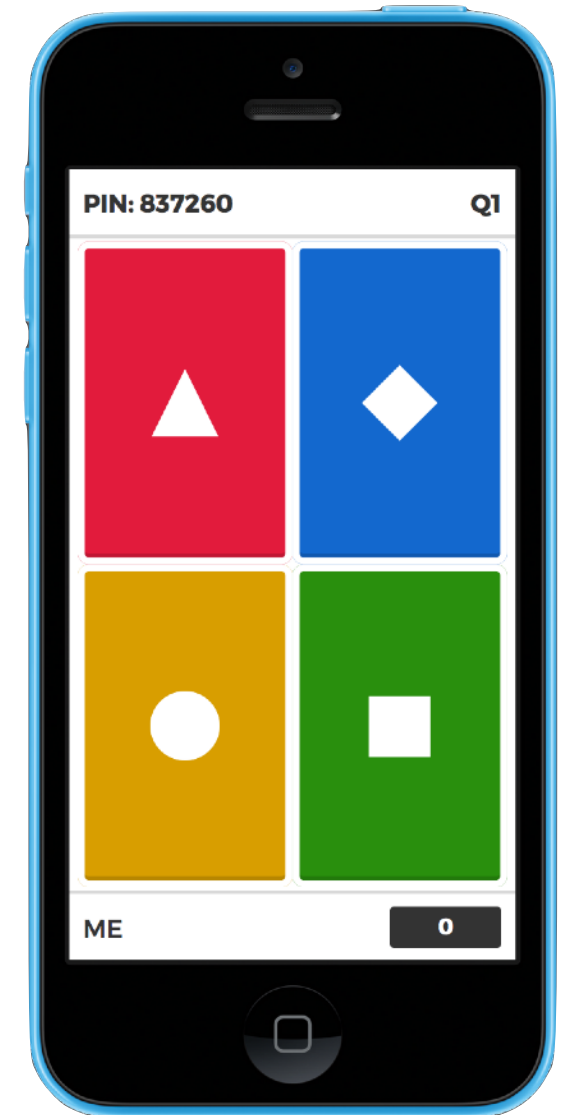
PIN  
eingeben



Nickname  
eingeben



mitspielen



1

2

3

4

